

Electronics Design with VHDL Selected Exercise

University of Alcalá. Edificio Politécnico.

Department of Electronic. Área: Tecnología Electrónica

01/09/2024

Some exercises are obtained from the collection of Electronic Design
Degree in Telecommunication Technologies Engineering. University of Alcalá



IDENTIFIATORS.

Exercise: Indicate which of the following identifiers are valid, justifying the answer

```
Salida_1  
Entrada__2  
E0_  
dir_mem_4  
  
Modulo 1  
Out@4  
1In
```

To determine whether an identifier is valid in VHDL, certain rules must be followed:

1. An identifier must begin with a letter.
2. It can contain letters, digits and underscores.
3. It cannot contain special characters such as @, spaces, etc.
4. It cannot begin with a digit.
5. It cannot end with an underscore or have two consecutive underscores.

IDENTIFIERS.

Exercise: Indicate what type of literal each is and calculate its decimal equivalence where appropriate.

```
340_000
3.4E+4
34 ns
2#1011#
8#21#
16#C3#
B"1010_1010"
O"47"
X"4FC"
'B'
"Esto es un ejemplo"
```

REGISTERS.

Exercise: Indicate the types or sub-types of data that are incorrectly declared.

```
type estado is (marcha, pausa, paro);  
type byte is bit_vector(7 downto 0);  
subtype letras is character range 'A' to 'Z';  
type texto is file of string;  
subtype vector is array (0 to 7) of integer;  
type A is array (0 to 5) of bit_vector(7 downto 0);  
type B is array (positive range 6 downto 1) of bit;  
subtype C is correcto (3 downto 1);  
type D is array (range 0 to 8) of bit;
```

REGISTERS.

Proposal: Write statements corresponding to the following declarations and value assignments.

```
-Retardo, constante de tipo físico time que toma el valor de 5 ns

-Comando, Variable de un tipo enumerado llamado Direccion ('izda','dcha','arri', 'abaj'). Debe inicializarse con el valor 'arri'.

-Entrada, variable de subtipo VALOR (subtipo de los enteros en el rango 0 a 255)

-Z, variable booleana inicializada a TRUE

-Dir, señal de tipo array de 16 bits con el bit más significativo a la izquierda

-Memoria, variable del tipo Array_mem que se define como un array de 256 bytes de capacidad del tipo std_logic_vector(7 downto 0). Debe inicializarse con ceros

-R, señal de tipo record, compuesto de los siguientes elementos: clk de tipo bit, D bit_vector de 8 bits, Dato entero de rango ilimitado. Debe inicializarse a 0, 1010_1010, 128
```

OPERATORS.

Exercise: Detect possible errors in the following assignments

```
constant A:time:=7.5 sec;  
  
variable B,C:time;  
  
variable R,S:bit_vector(2 downto 0);  
  
variable T:bit_vector(3 downto 0);  
  
C:=A/3;  
  
C:=A+1;  
  
C:=B/A*C;  
  
T(0 to 2):=R;  
  
T:=R&1;  
  
S:=S and not R;  
  
S:=R & S;
```

OPERATORS.

Proposal: Indicate the data type of the result of the following operations and its value

```
Variable A: integer:=25;

Constant B: integer:=4;

Signal C:bit_vector(0 to 3):="1001";

Variable D:=time:= 10 us

Signal E:bit_vector(0 to 5);

D:=D*(B-1);

E <= "10"&C;

A:=B;

C <= C sra 2;
```

Determine if the following assignments and arithmetic operations are correct.

```
Signal std1: std_logic_vector(3 downto 0);
```

```
Signal uns1: unsigned(3 downto 0);
```

```
uns1  <=  unsigned(std1);
```

```
std1  <=  std_logic_vector(uns1);
```

```
uns1  <=  to_unsigned(3,4);
```

```
std1  <=  std_logic_vector(to_unsigned(3,4));
```

Determine if the following assignments and arithmetic operations are correct.

```
Signal std1,  std2,  std3: std_logic_vector(3    downto 0);  
Signal uns1,  uns2,  uns3: unsigned(3    downto 0);
```

```
uns1  <=  uns2+uns3;
```

```
std1  <=  std2+std3;
```

```
std1  <=  std2+1;
```

```
std1 <= std_logic_vector(unsigned(std2)+unsigned(std3));
```

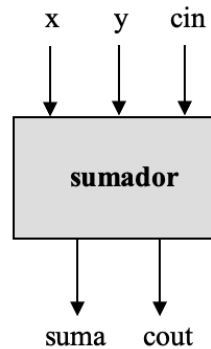
```
std1  <=      std_logic_vector(unsigned(std2)+3);
```

BASIC UNIT

Exercise: Design a one-bit binary adder that corresponds to the entity shown in Figure

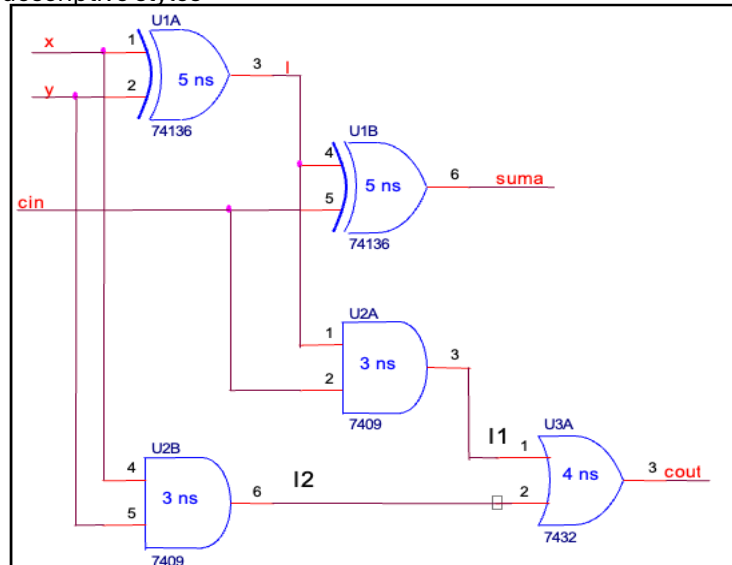
Definition of basic components:

```
entity sumador is
  port (
    x,y, cin: in bit;
    suma, cout: out bit);
end sumador;
```



Component architectures:

Knowing that your hardware implementation using basic logic gates is as shown in Figure, where the delay time introduced by each of them is indicated, implement 3 different architectures for the adder using the structural, dataflow and algorithmic descriptive styles

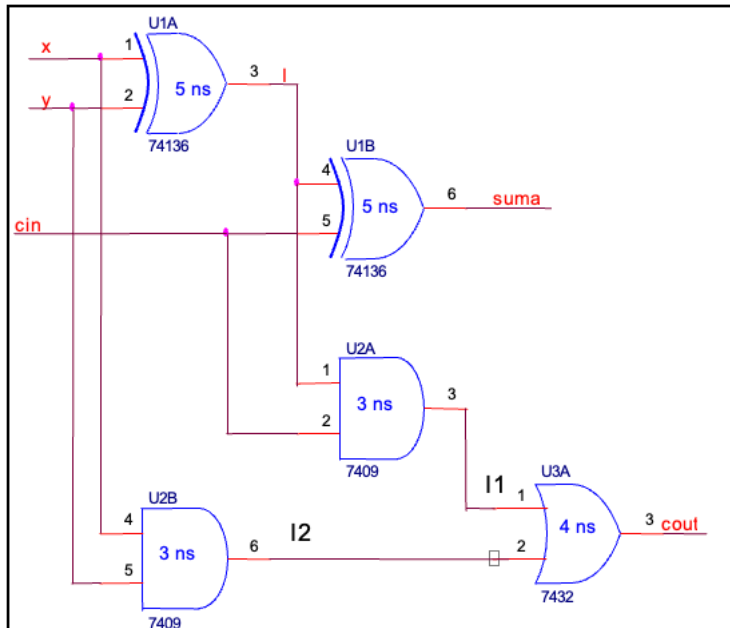
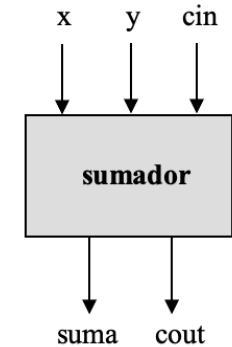


BASIC UNIT

Exercise (CONT.)

2. Data Flow Style: In the dataflow style, the adder is modelled directly using the logic equations that describe its behaviour. The delays here are associated with the propagation paths of the data flow.

```
entity sumador is
port (
    x,y, cin: in bit;
    suma, cout: out bit);
end sumador;
```



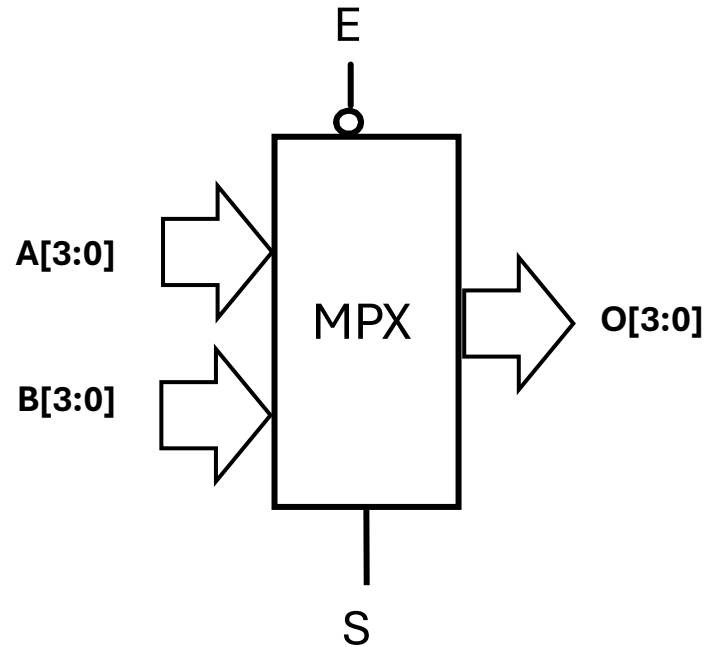
BASIC UNIT

Exercise (CONT.)

3. Algorithmic Style: The algorithmic style describes the behaviour of the adder at a higher level of abstraction, without concern for the specific hardware structure. Delays are global and are applied between inputs and outputs.

BASIC UNIT

Design a Multiplexer of 2 channels and 4 bits



E	S	O
1	X	0
0	0	A
0	1	B

Process.

Write an Example of Signal assignment

```
library ieee;
use ieee.std_logic_1164.all;

entity SignalExample is
  port (
    clk : in std_logic;
    reset : in std_logic;
    output_signal : out std_logic
  );
end SignalExample;

architecture Behavioral of SignalExample is
  signal internal_signal : std_logic := '0';
begin
  process(clk, reset)
  begin
    if reset = '1' then
      internal_signal <= '0';
    elsif rising_edge(clk) then
      internal_signal <= not internal_signal;
    end if;
  end process;

  output_signal <= internal_signal;
end Behavioral;
```

Process.

Write an example of Signal assignment and determine the incorrect option

Different options for implementing multiplexor of 2 input

```
process(e1,e2, sel)
begin
  if (sel='0') then
    o1<= e1;
  else
    o1<= e2;
  endif;
end process;
```

```
process(e1,e2, sel)
begin
  o1<= e1;
  if (sel='1') then
    o1<= e2;
  endif;
end process;
```

```
process(e1,e2, sel)
begin
  if (sel='1') then
    o1<= e2;
  endif;
  o1<= e1;
end process;
```

BASIC UNIT

Exercise: Desing a 4 bit Adder

```
-- Declaración de la entidad
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity Sumador4Bits is
    Port (
        A      : in  STD_LOGIC_VECTOR (3 downto 0);  -- Entrada A (4 bits)
        B      : in  STD_LOGIC_VECTOR (3 downto 0);  -- Entrada B (4 bits)
        SUM     : out STD_LOGIC_VECTOR (3 downto 0);  -- Salida Suma (4 bits)
        Carry_Out : out STD_LOGIC                    -- Carry de salida
    );
end Sumador4Bits;
```

Complete the architecture

Arquitectura del sumador

BASIC UNIT

Exercise: Write an example of a combinational design. Driver of a 7 LED display

```
LIBRARY IEEE;
USE IEEE.std_logic_1164.all;
ENTITY bit2led IS
    PORT (
        bits: IN STD_LOGIC_VECTOR (3 downto 0);
        led: OUT STD_LOGIC_VECTOR (6 downto 0)
    );
END bit2led;
ARCHITECTURE bit2led_arch OF bit2led IS
BEGIN
    -- contenido de la arquitectura
END bit2led_arch;
```

Concurrent sentences.

1) Example with WHEN ELSE

```
Señal  D <= Exp1 WHEN condition ELSE Exp 2;
```

```
Señal  D <= Exp1 WHEN Cond1 ELSE  
  
      Exp2          WHEN      Cond2          ELSE  
      .Exp3..... WHEN      Cond3          ELSE  
      Expression N      WHEN      Cond  
      ELSE      Expresión;
```

CANNOT BE USED WITHIN A PROCESS

```
library      IEEE;  
      use      IEEE.std_logic_1164.all;  
entity      mux21      is  
port ( e1,      e2:      IN std_logic;  
      sel:      IN std_logic;  
      o1:      OUT std_logic);  
end      mux21;  
architecture      A of mux21  
      is begin  
      o1 <= e1 when sel='0' else e2;  
end      A;
```

Concurrent sentences.

2) Example WITH ... SELECT ...

```
WITH exp_selection SELECT
Signal Des    <=
    Exp1      WHEN    Valor_Sel1, Exp2 WHEN
    Valor_Sel2,
    ...
    Exp N    WHEN    OTHERS;
```

```
WITH sel SELECT
o1 <=
    e1 WHEN '0' ,
    e2 WHEN OTHERS;
```

CANNOT BE USED WITHIN A PROCESS

3) Example IF THEN ELSIF ELSE

```
IF cond      1 THEN
    Con      1;
ELSIF cond   2 THEN
    Con      2;
ELSIF ... THEN
    ...
ELSE Con;
END IF;
```

```
architecture A of mux21 is
begin
    process(e1,e2, sel)
    begin
        if (sel='0') then
            o1 <= e1;
        else
            o1<= e2;
        endif;
    end process;
end A;
```

Concurrent sentences.

4. Example CASE WHEN

```
CASE    objet    IS
      WHEN    case1    =>
              Cod1;

      WHEN    case2    =>
              Cod2;

      WHEN    case3    =>
              Cod3;

      WHEN    OTHERS    =>
              Cod;

END CASE;
```

Cannot be used outside the process

```
architecture    A    of    mux21    is begin
  process(e1,e2,    sel)
  begin
    case    sel    is
      when    '0'    =>
              o1<=e1;

      when    others    =>
              o1<=e2;

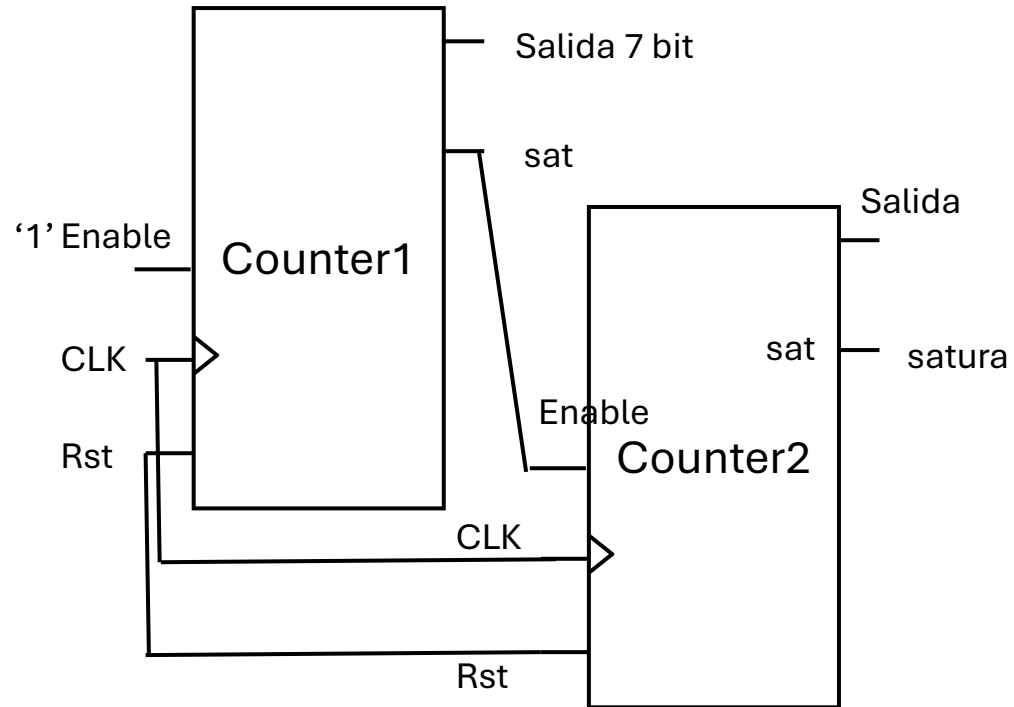
    end case; end
  process;
end A;
```

```
architecture Behavioral of ExampleCase is
  signal sel : std_logic_vector(1 downto 0);
  signal result : std_logic_vector(3 downto 0);
begin
  process(sel)
  begin
    case sel is
      when "00" =>
        result <= "0001";
      when "01" =>
        result <= "0010";
      when "10" =>
        result <= "0100";
      when "11" =>
        result <= "1000";
      when others =>
        result <= "0000"; -- Default case
    end case;
  end process;
end Behavioral;
```

WHEN OTHERS is mandatory if not all cases are covered

Section ARCHITECTURE.

Example of Component interconexion



Objetive

```
component nombre
    generic (definition of generics);
    port (definition of ports);
end component;
```

```
nombre_instancia : component name
    GENERIC MAP (.....)
    PORT MAP (.....);
```

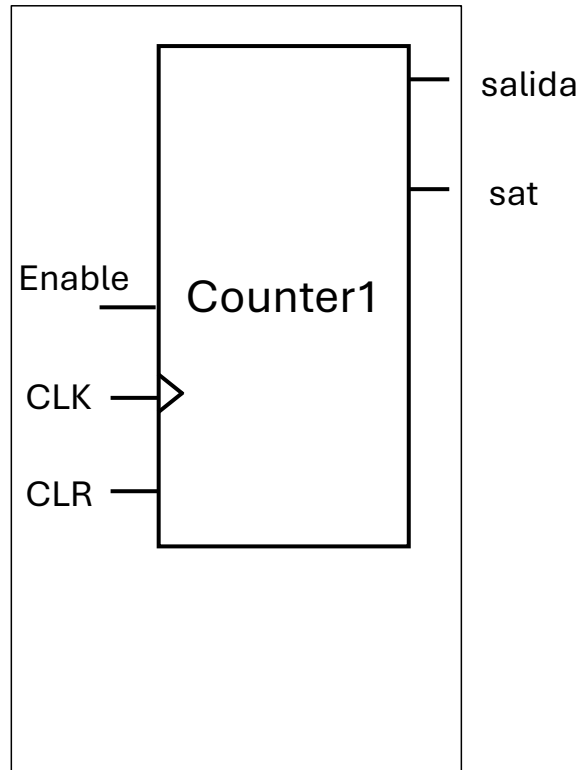
Component declaration.

In the area of signal and component declarations

Component usage.

Within the body of the architecture (BEGIN)

Section ARCHITECTURE. Component interconexion Example



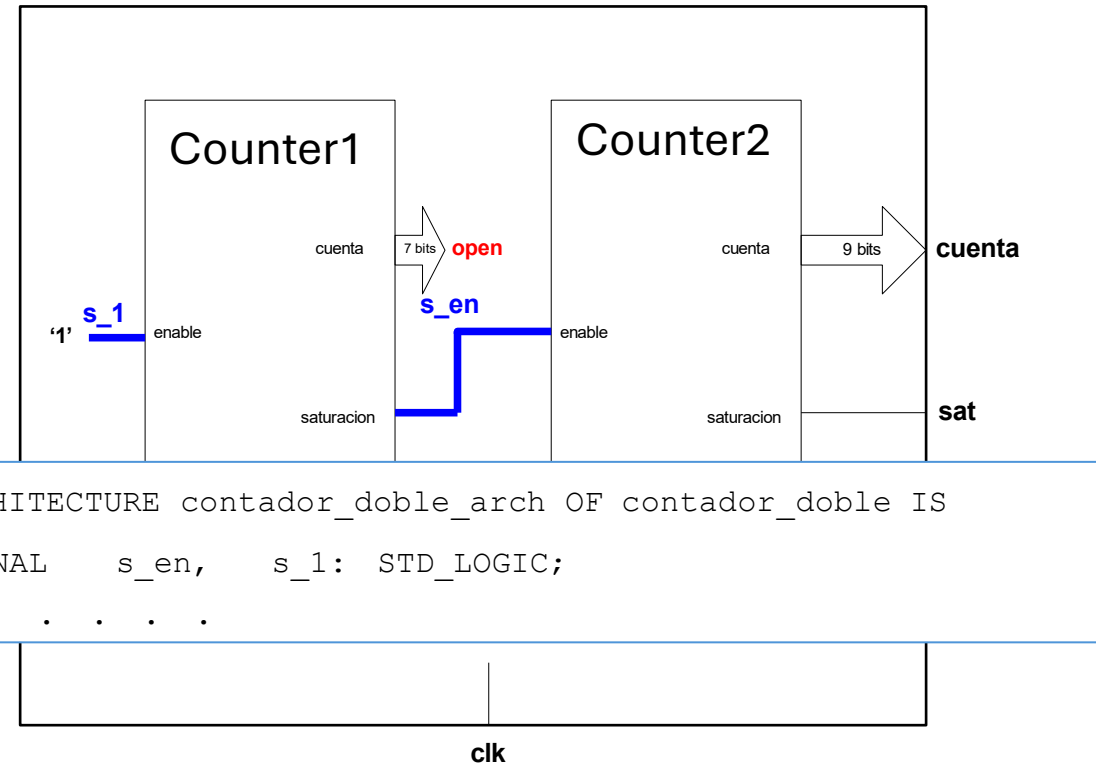
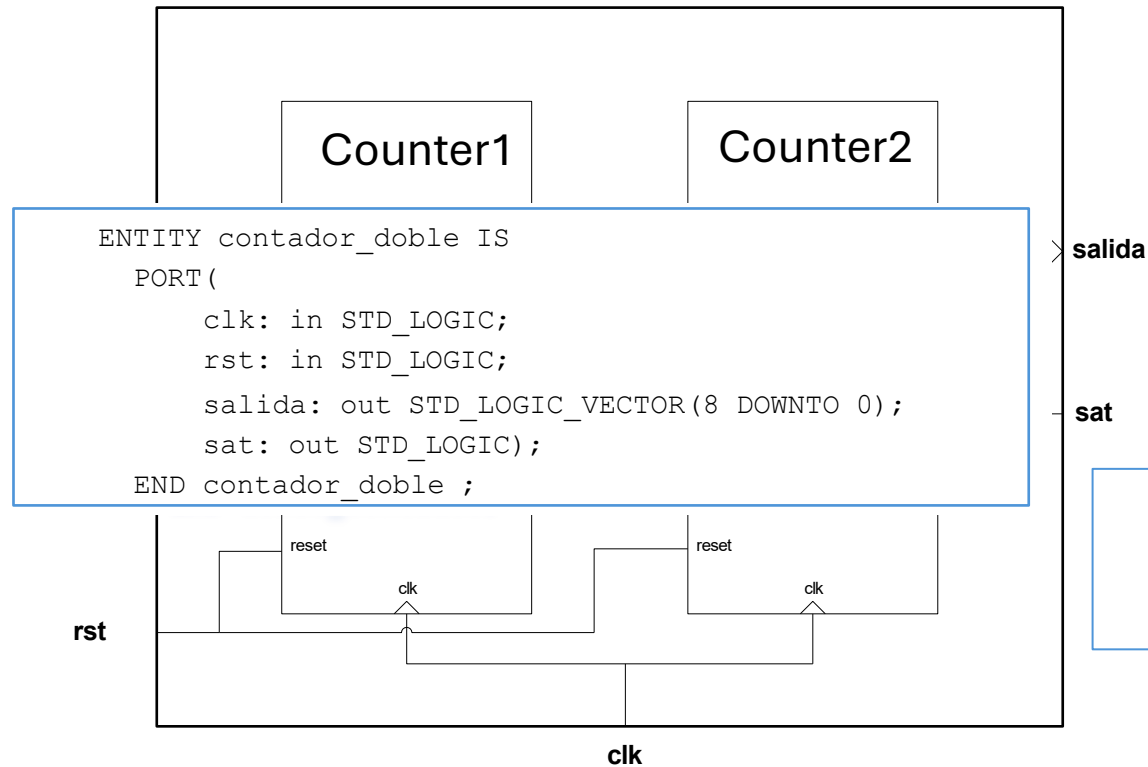
```
ENTITY contador IS
    GENERIC (N:integer:=10);
    PORT(
        clk: in STD_LOGIC; reset: in STD_LOGIC;
        enable: in STD_LOGIC;
        salida: out STD_LOGIC_VECTOR(N-1 DOWNT0 0);
        sat: out STD_LOGIC);
END contador;

ARCHITECTURE contador_arch OF contador IS
BEGIN
    . . . . .
END contador;
```

We must first have designed the lower hierarchy level.

Section ARCHITECTURE. Component interconexión

Example

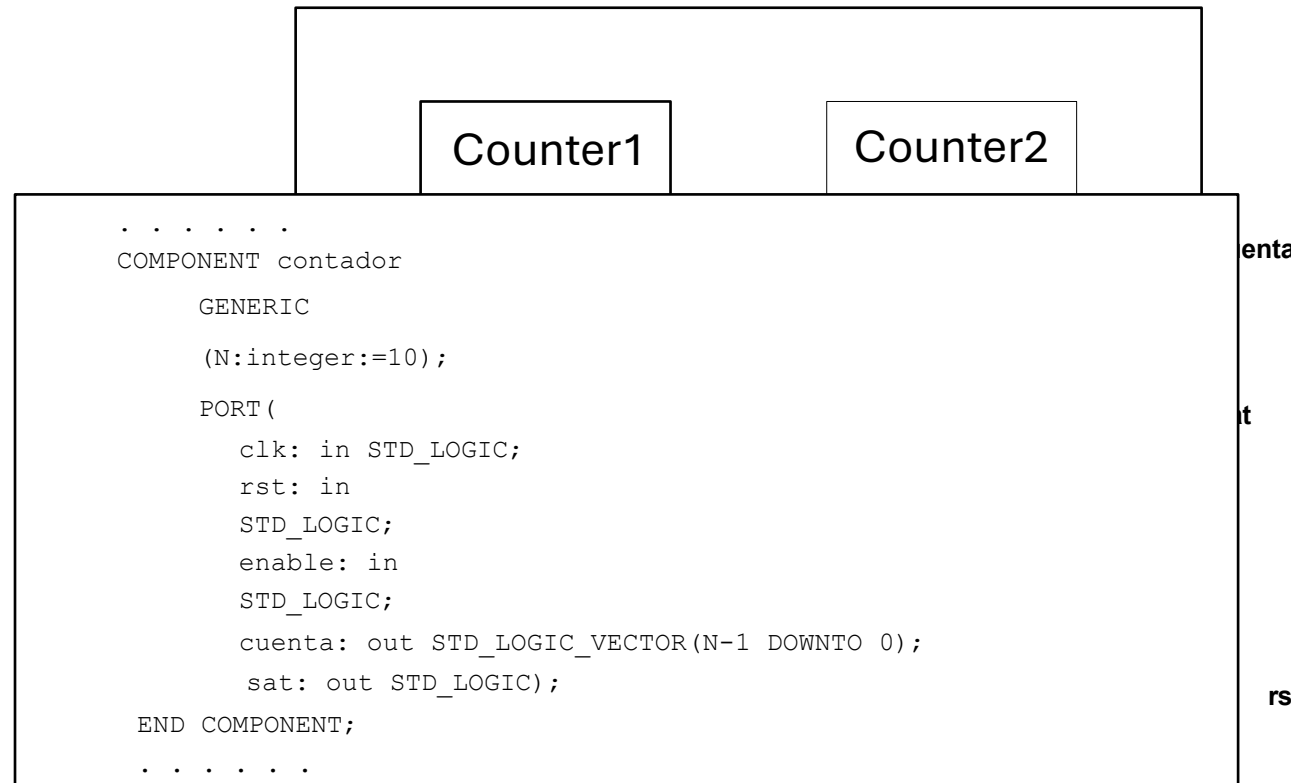


STEP 1: Define the entity of DOUBLE_COUNTER

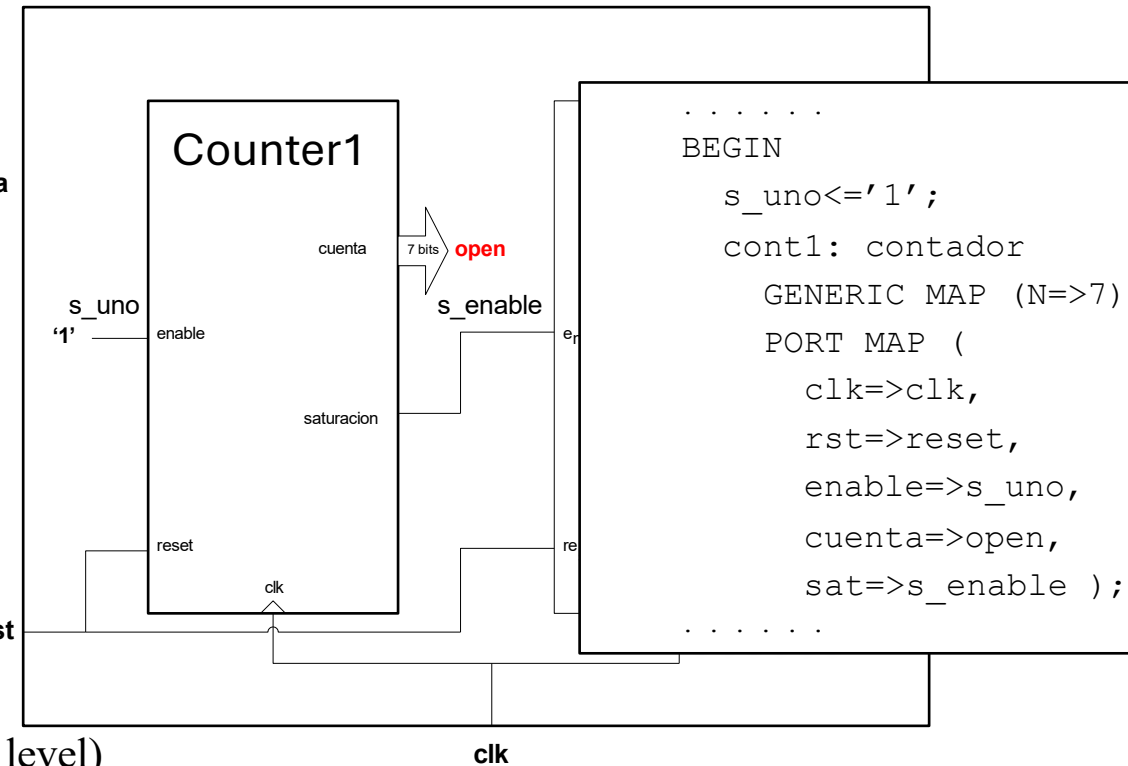
STEP 2: Declare the necessary signals (cables not going outside)

Section ARCHITECTURE. Component interconexion

Example



STEP 3: Declare the necessary components (entities at the lower hierarchy level)



STEP 4: Make the instance (call) to the first counter (inside the BEGIN).

Attributes

An attribute is an associated characteristic to an element (data type, signal, entity,...) that provides additional information.

Attribute $\neq$ Value

An object has a single value and can have multiple attributes.

VHDL provides a number of predefined attributes.

```
name element'Name attribute
```

```
Byte_a'range  
Signal a'event
```

ATTRIBUTES.

Exercise: Given the following signal declaration, indicate the values they return depending on the attributes applied.

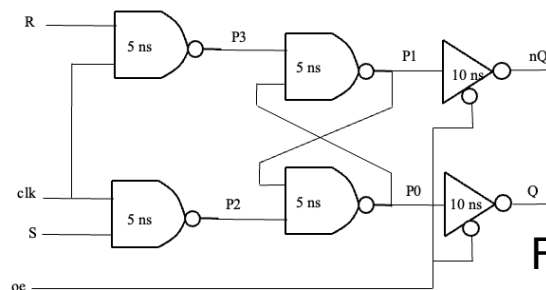
```
Type tabla is array (1 to 5, 7 downto 0) of bit;  
Signal linea: bit_vector(15 downto 0);  
Signal matriz:tabla;  
Linea'left  
Linea'right(1) o linea'right  
Matriz'left(1) o matriz'left  
Matriz'right(2)  
Matriz'left(2)  
Matriz'high(1)  
Linea'low
```

```
Matriz'ascending(2)  
Matriz'length(2)  
Linea'range  
Linea'reverse_range  
Tabla'base  
Type vocales is ('a','e','i','o','u');  
Type DiaMes is integer range 1 to 31;  
DiaMes'high  
Vocales'pos('i')  
Vocales'val(1)  
Vocales'base  
Signal reloj: bit;  
If reloj'event and clk='1'  
If reloj'stable(2ns)
```

ATTRIBUTES. Proposal:

It is desired to design a VHDL model of a high level active RS latch with tri-state output control according to the schematic in Figure. For this purpose, a structural description level based on NAND and NOT logic gates described at RTL or data stream level shall be used. All input-output signals shall be defined as `std_logic`. The test bench shall be modelled by two processes, one to generate a square clock signal of period 30 ns and one to generate the other stimulus signals with the following intervals: r: pulse high level between 65-95 ns and 155-185 ns, s: pulse high level between 10-40 ns and 155-185 ns, oe: pulse high level between 140-155 ns. The simulation time shall be 250 ns.

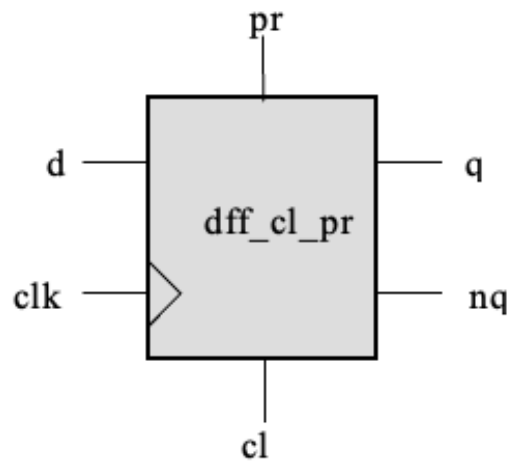
```
architecture flujo_datos of p_nand is
begin
    y<= (a nand b) after 5 ns;
end flujo_datos;
-- Modelado del latch RS
library ieee;
use ieee.std_logic_1164.all;
entity latch_rs is
    port (r, s, clk, oe : in std_logic;
          q, nq : out std_logic);
end latch_rs;
```



Figure

ATTRIBUTES. Proposal:

Design a D-type flip-flop active on rising edge and with asynchronous PRESET (PR) and CLEAR (CL) signals according to the truth table in Figure. A model entity named `dff_cl_pr` is provided where the input-output ports and the delays to be applied are defined. The architecture must be defined at the algorithmic level and it is desired to introduce some controls of the setup and hold time by means of the assert sentence with a severity level of warning. It is also desired to control that the two asynchronous signals are not activated at the same time, generating in this case a note type severity level. Design a test bench to demonstrate the proper functioning of the circuit and the activation of the imposed simulation controls.



pr	cl	clk	d	q	nq
1	x	x	x	1	0
0	1	x	x	0	1
0	0	↑	0	0	1
0	0	↑	1	1	0
0	0	0 → 1	x	q0	nq0

Figure

SIGNAL AND VARIABLES

Exercise: Given the architectures "exemp1" and "exemp2" defined for the entity "question" complete the attached simulation graph for the given values of D and CLK.

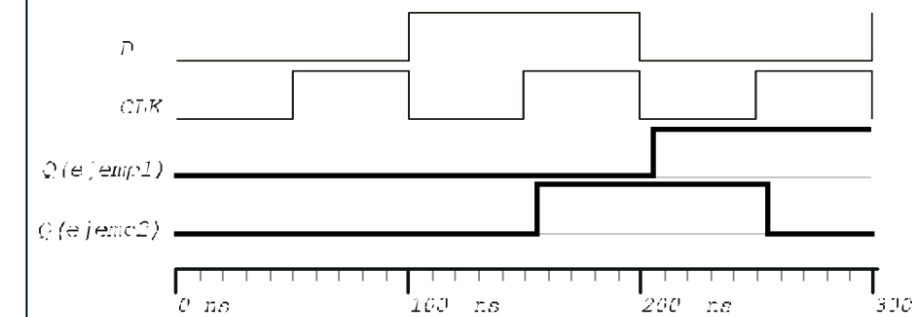
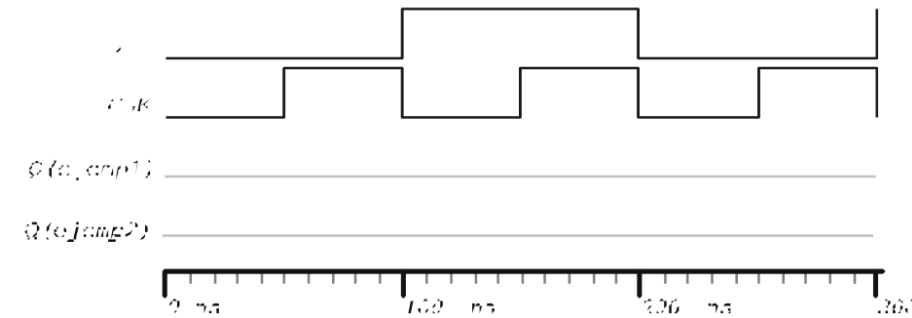
```
entity cuestion is
    port(D,CLK: in bit; Q:out bit);
    constant retardo:time:=5 ns;
end cuestion;
```

```
architecture ejemp1 of cuestion is
    signal estado:bit;
begin
    process(CLK)
    begin
        if CLK='1' and CLK'event then
            estado <=D;
        end if;
        Q<=estado after retardo;
    end process;
end ejemp1;
```

```
architecture ejemp2 of cuestion is
begin
    process(CLK)
    variable estado:bit;
    begin
        if CLK='1' and CLK'event then
            estado:=D;
        end if;
        Q<=estado after retardo;
    end process;
end ejemp2;
```

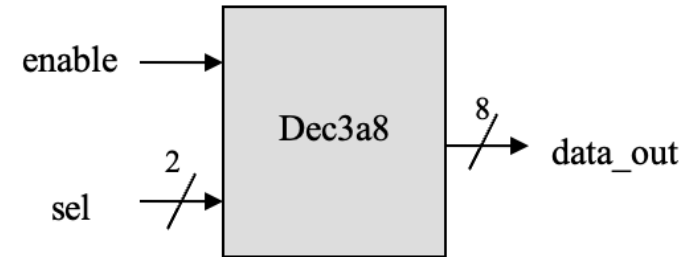
ejemp1

ejemp2



CASE. Exercise: We want to design a 3 to 8 decoder according to the schematic shown in Figure, where all the inputs-outputs are of type `std_logic`. The enable signal is active high and in case the decoder is not enabled all outputs must be disabled (at level 0). Make two models for it, one with sequential statements and the other with concurrent statements. Also implement a test bench that generates an enable signal at level 0 until the 30 ns instant of simulation and that goes to level 1 until the end of the simulation. For the sel signal, generate periodic sequences between "000" and "111" that change every 10 ns, starting at "000".

```
library ieee;
use ieee.std_logic_1164.all;
entity dec3a8 is
    port (enable: in std_logic;
          sel: in std_logic_vector(2 downto 0);
          dato_out: out std_logic_vector(7 downto 0));
end dec3a8;
```



GENERATE.

Exercise: Implement a 4-bit comparator using a structural descriptive style based on the 1-bit comparators, as shown in Figure 20.1. Define a constant n to define the size of the comparator that allows the comparator to be modified only by changing the value of the constant. Design a test bench that generates as stimuli various combinations of values for a and b to check the correct operation of the comparator. The control inputs gt , eq and lt shall be fixed during the simulation with the following values ($gt='0'$, $eq='1'$, $lt='0'$).

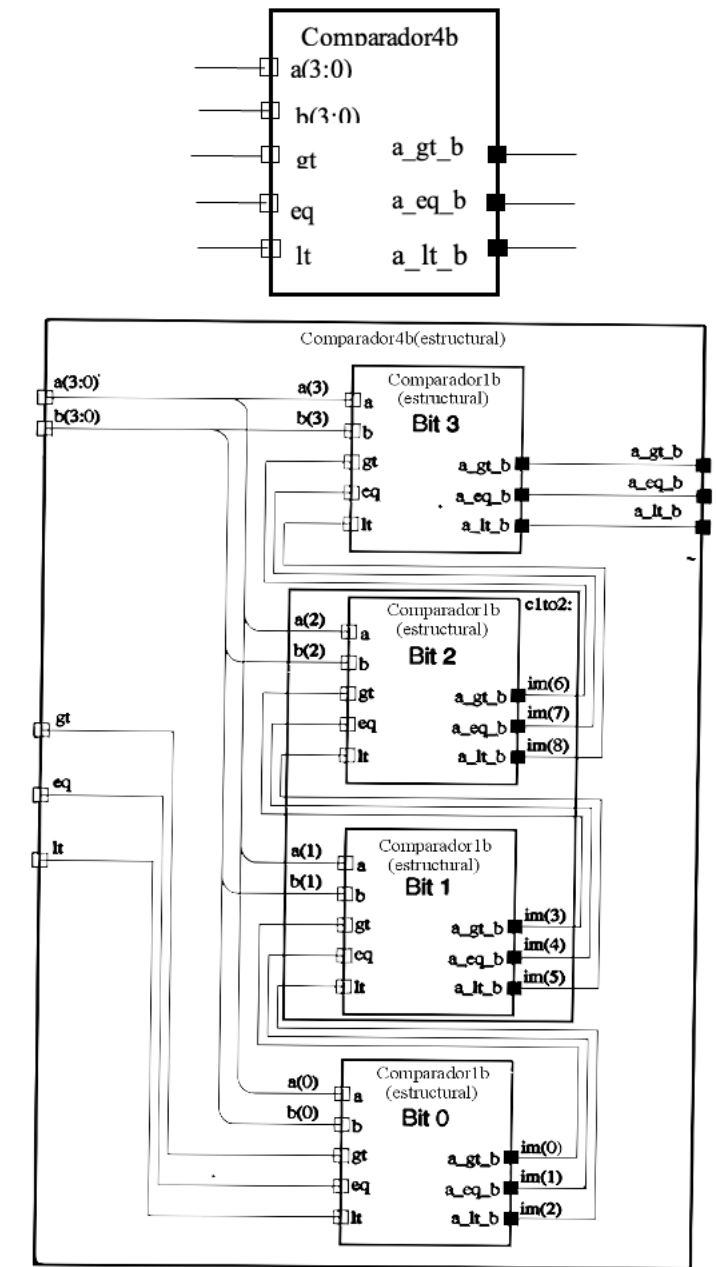
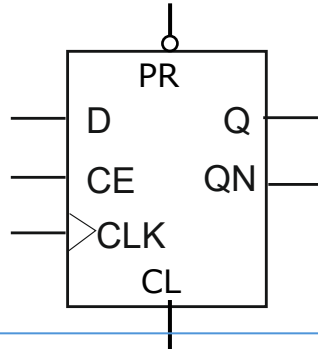


Figura 20.1. Descripción VHDL del comparador de cuatro bits

Example: Design a bistable with synchronous Inputs



The sensitivity list consists of the clock signal and the asynchronous signals.

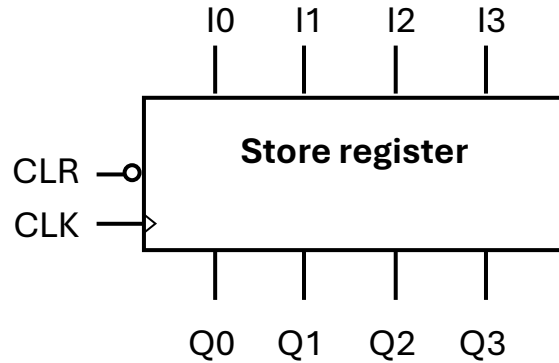
CL	PR	CE	D	CLK	Q_{t+1}
1	X	X	X	X	0
0	0	X	X	X	1
0	1	0	X	$\downarrow$	Q_t
0	1	1	0	$\downarrow$	0
0	1	1	1	$\downarrow$	1
0	1	X	X	0	Q_t
0	1	X	X	1	Q_t

```
library ieee;
use ieee.std_logic_1164.all;
entity ff_d_PC is
  port (
    CLK : in  std_logic;
    D    : in  std_logic;
    PR   : in  std_logic;
    CL   : in  std_logic;
    CE   : in  std_logic;
    Q    : out std_logic);
end ff_d_PC;
architecture rtl of ff_d_PC is
begin
  process (CLK, PR, CL)
  begin
    if CL = '1' then
      Q <= '0';
    elsif PR = '1' then
      Q <= '1';
    elsif CLK'event and CLK = '1' then
      if CE = '1' then
        Q <= D;
      end if;
    end if;
  end process;
end rtl;
```

1st, asynchronous operation is modelled.

2nd synchronous operation is modelled

Example: Design a Registers - Storage



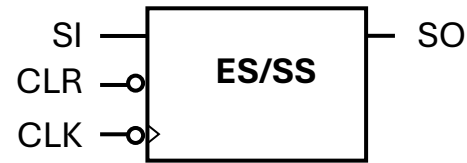
Modes of operation	Inputs			Outputs
	$\overline{\text{CLR}}$	CLK	Ii	$Q_{i,t+1}$
Reset (clear)	0	X	X	0
Load "1"	1	↑	1	1
Load "0"	1	↑	0	0

```
library ieee;
use ieee.std_logic_1164.all;

entity reg_alma is
    port(
        I    : in  std_logic_vector(3 downto 0);
        CLK  : in  std_logic;
        CLR  : in  std_logic;
        Q     : out std_logic_vector(3 downto 0));
end reg_alma;
```

```
architecture RTL of reg_alma is
begin
    process(CLK, CLR)
    begin
        if CLR = '0' then
            Q <= (others => '0');
        elsif (CLK'event and CLK = '1') then
            Q <= I;
        end if;
    end process;
end RTL;
```

Example: Design a Serial input - serial output



Inputs			Señales intermedias				Output
CLR	CLK	SI	$Q0_{t+1}$	$Q1_{t+1}$	$Q2_{t+1}$	$Q3_{t+1}$	SO
0	X	X	0	0	0	0	0
1	0 o 1	X	$Q0_t$	$Q1_t$	$Q2_t$	$Q3_t$	$Q3_t$
1	↓	si	si	$Q0_t$	$Q1_t$	$Q2_t$	$Q2_t$

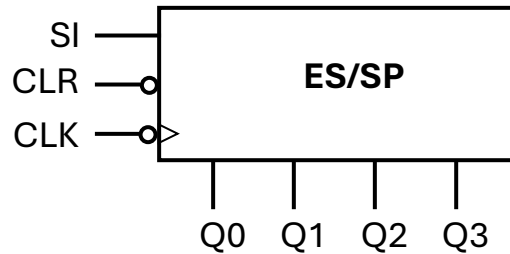
```
library ieee;
use ieee.std_logic_1164.all;

entity reg_es_ss is
    port(
        SI : in  std_logic;
        CLR : in  std_logic;
        CLK : in  std_logic;
        SO : out std_logic);
end reg_es_ss;
```

```
architecture RTL of reg_es_ss is
    signal S : std_logic_vector(0 to 3);
begin
    process(CLR, CLK)
    begin
        if CLR = '0' then
            S <= (others => '0');
        elsif (CLK'event and CLK = '0') then
            S <= SI & S(0 to 2);
        end if;
    end process;
    SO <= S(3);
end RTL;
```

Concatenation is used to group elements and form vectors.

Example: Design a Serial input - paralel output



Inputs			Outputs			
CLR	CLK	SI	$Q0_{t+1}$	$Q1_{t+1}$	$Q2_{t+1}$	$Q3_{t+1}$
0	X	X	0	0	0	0
1	0 or 1	X	$Q0_t$	$Q1_t$	$Q2_t$	$Q3_t$
1	↓	si	si	$Q0_t$	$Q1_t$	$Q2_t$

```
library ieee;
use ieee.std_logic_1164.all;

entity reg_es_sp is
  port(
    SI  : in  std_logic;
    CLR : in  std_logic;
    CLK : in  std_logic;
    Q   : out std_logic_vector(0 to 3));
end reg_es_sp;
```

```
architecture RTL of reg_es_sp is
  signal S : std_logic_vector(0 to 3);
begin
  process (CLR, CLK)
  begin
    if CLR = '0' then
      S <= (others => '0');
    elsif (CLK'event and CLK = '0') then
      S <= SI & S(0 to 2);
    end if;
  end process;
  Q <= S;
end RTL;
```

Example: Design universal shift register

```
library ieee;
use ieee.std_logic_1164.all;

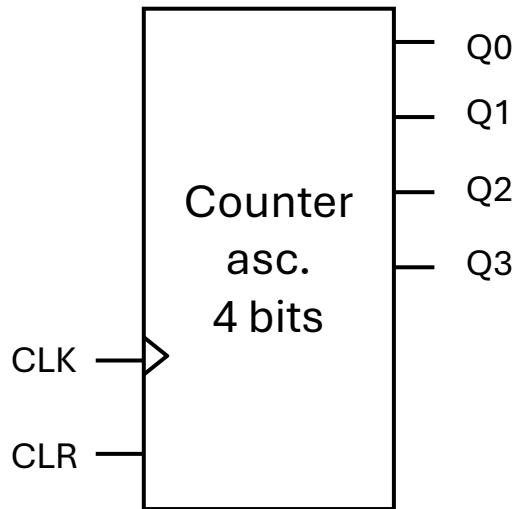
entity reg_uni is
  port(
    I   : in  std_logic_vector(0 to 3);
    SRI : in  std_logic;
    SLI : in  std_logic;
    C1  : in  std_logic;
    C0  : in  std_logic;
    CLR : in  std_logic;
    CLK : in  std_logic;
    Q   : out std_logic_vector(0 to 3));
end reg_uni;
```

```
architecture RTL of reg_uni is
  signal S : std_logic_vector(0 to 3);
begin
  process(CLR, CLK)
    variable sel : std_logic_vector(1 downto 0);
  begin
    if CLR = '1' then
      S <= (others => '0');
    elsif (CLK'event and CLK = '1') then
      sel := C1 & C0;
      case sel is
        when "01" => S <= SRI&S(0 to 2);
        when "10" => S <= S(1 to 3)&SLI;
        when "11" => S <= I;
        when others => null;
      end case;
    end if;
  end process;
  Q <= S;
end RTL;
```

Example: Counters - Synchronous

Design an ascending, binary, 4-bit, synchronous:

CLR	CLK	Q_{t+1}
1	X	0
0	↑	$Q_t + 1$



```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_unsigned.all;

entity cont_asc is
    port(
        CLK : in  std_logic;
        CLR  : in  std_logic;
        Q    : out std_logic_vector(3 downto 0));
end cont_asc;
```

```
architecture RTL of cont_asc is
    signal Pre_Q : std_logic_vector(3 downto 0);
begin
    process(CLK, CLR)
    begin
        if CLR = '1' then
            Pre_Q <= (others => '0');
        elsif (CLK'event and CLK = '1') then
            Pre_Q <= Pre_Q + 1;
        end if;
    end process;
    Q <= Pre_Q;
end RTL;
```

Example

Design a 4-bit up counter with asynchronous clk, synchronous load

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

-- Entidad del contador
entity contador_4bits is
    Port (
        clk : in STD_LOGIC;          -- Señal de reloj asíncrono
        reset : in STD_LOGIC;         -- Señal de reset
        load : in STD_LOGIC;          -- Señal de carga síncrona
        load_val : in STD_LOGIC_VECTOR(3 downto 0); -- Valor a cargar
        count : out STD_LOGIC_VECTOR(3 downto 0) -- Salida del
    contador
    );
end contador_4bits;
```

```
-- Arquitectura del contador
architecture Behavioral of contador_4bits is
    signal counter: STD_LOGIC_VECTOR(3 downto 0) := "0000"; -- Señal int
begin
    -- Proceso asíncrono para el reloj
    process(clk, reset)
    begin
        if reset = '1' then
            counter <= "0000"; -- Reinicia el contador a 0 en caso de reset
        elsif rising_edge(clk) then
            if load = '1' then
                counter <= load_val; -- Carga el valor si load está activo
            else
                counter <= counter + 1; -- Incrementa el contador
            end if;
        end if;
    end process;

    -- Asigna el valor del contador a la salida
    count <= counter;
end Behavioral;
```

Proposal: Design an 8-bit synchronous counter circuit at the algorithmic level with the possibility of parallel load, reset, enable and count-up/count-down, where all input-output signals are of type std_logic, its entity corresponds to the schematic in Figure 1 and its behaviour is as shown in Figure 2.

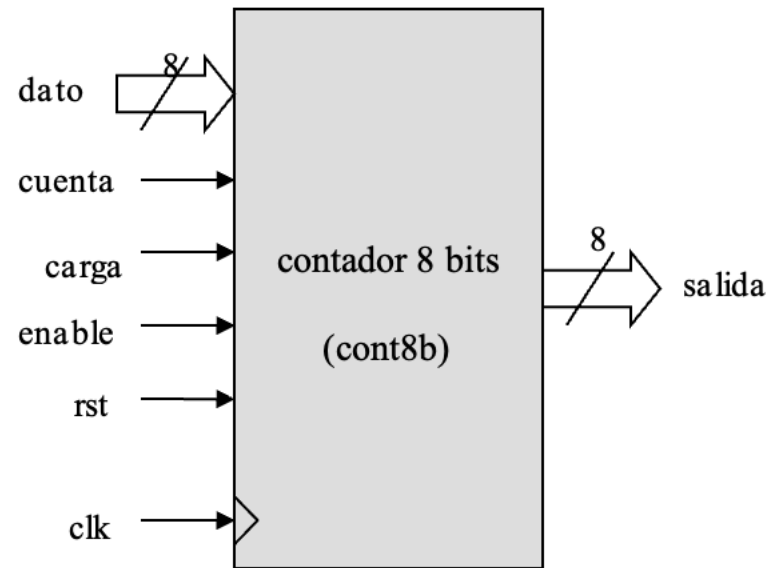


Figure 1

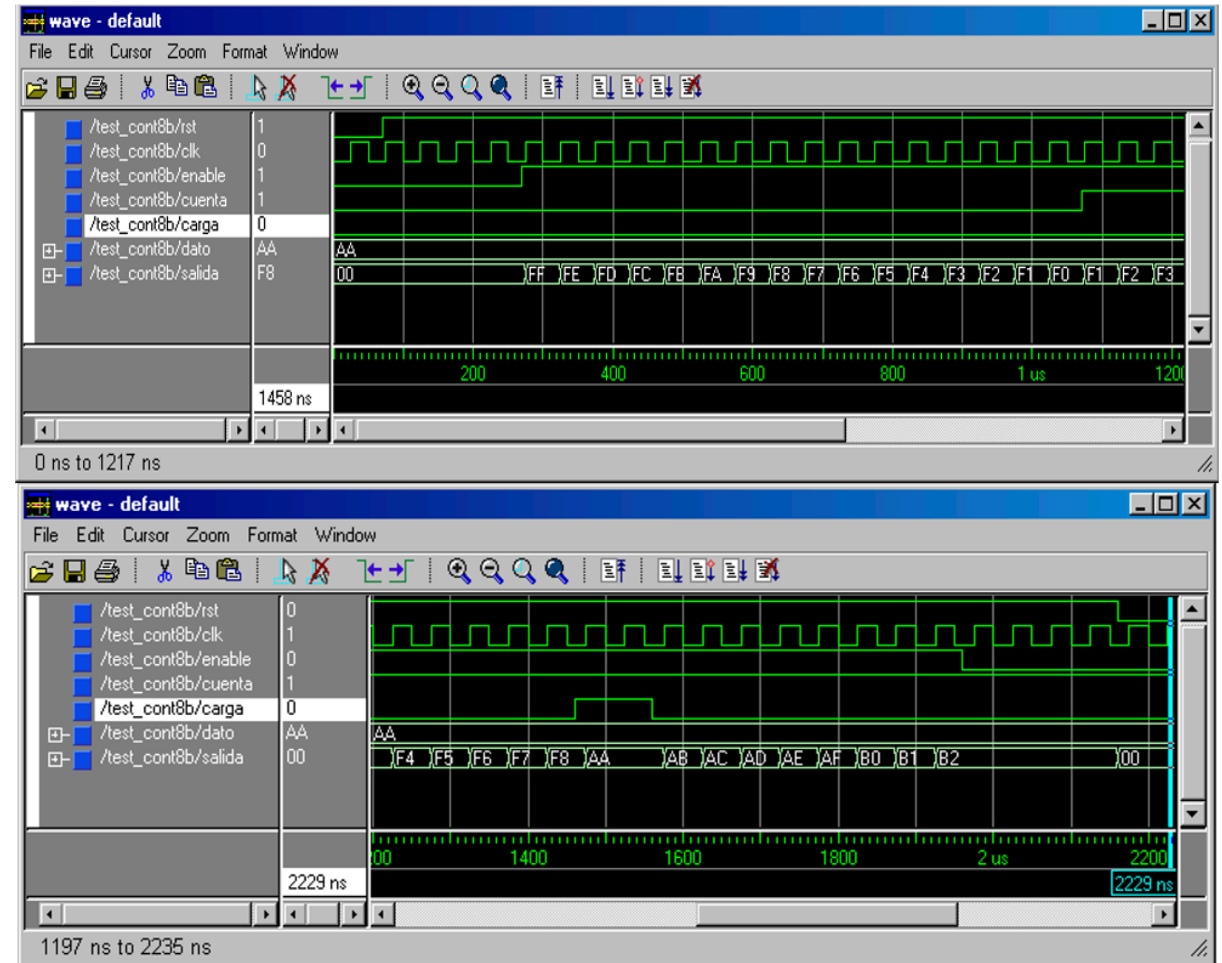
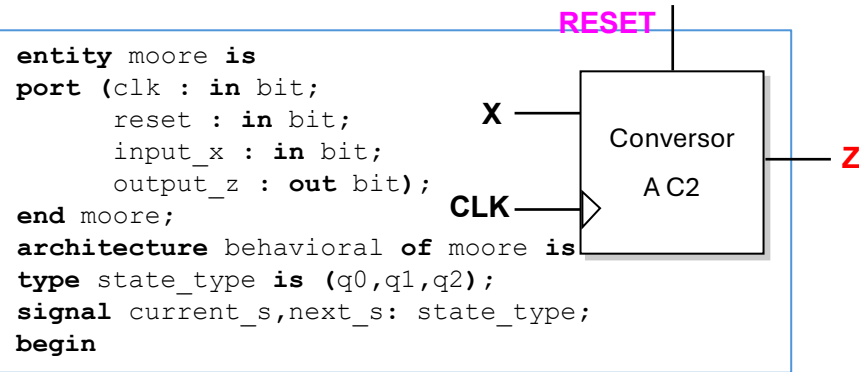


Figure 2

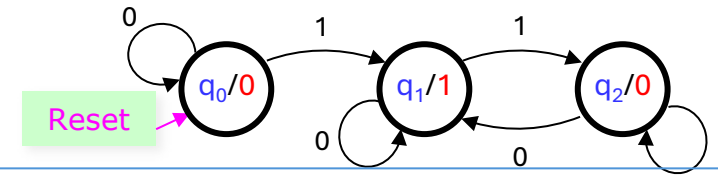
Coding in VHDL: Design a Moore State Machine



```

--combinational process to model next-state-logic
process (current_s,input_x)
begin
  case current_s is
    when q0 => --when current state is "q0"
      if(input_x = '0') then next_s <= q0;
      else next_s <= q1;
      end if;
    when q1 => --when current state is "q1"
      if(input_x = '0') then next_s <= q1;
      else next_s <= q2;
      end if;
    when q2 => --when current state is "q2"
      if(input_x = '0') then next_s <= q1;
      else next_s <= q2;
      end if;
  end case;
end process;

```



```

--sequential process to describe current-state-logic
process (clk,reset)
begin
  if (reset='1') then
    current_s <= q0; --asynchronous reset
  elsif (clk'event and clk='1') then
    current_s <= next_s; --state change.
  end if;
end process;

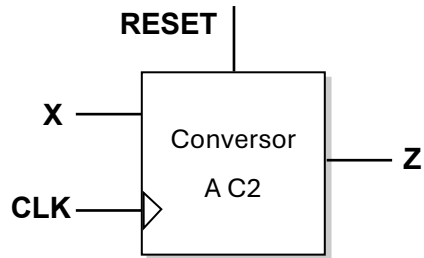
```

```

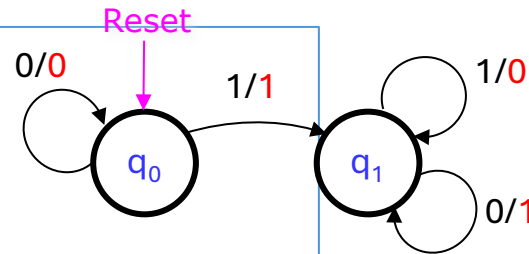
process (current_s) --combinational process to model output-logic
begin
  case current_s is
    when q0 => output_z <= '0'; --when current state is "q0"
    when q1 => output_z <= '1'; --when current state is "q1"
    when q2 => output_z <= '0'; --when current state is "q2"
  end case;
end process;
end behavioral;

```

Coding in VHDL: Design a Mealy State Machine



```
entity mealy is
port (clk : in bit;
      reset : in bit;
      input_x : in bit;
      output_z : out bit);
end mealy;
```



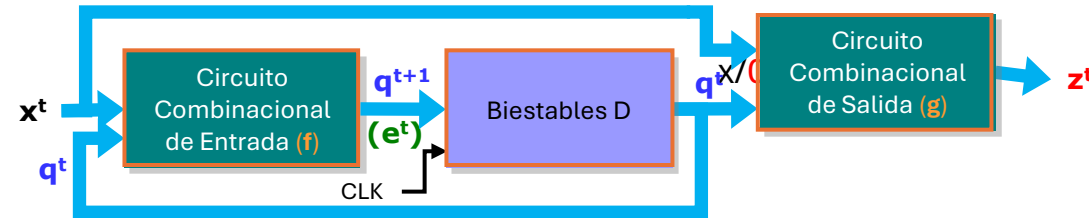
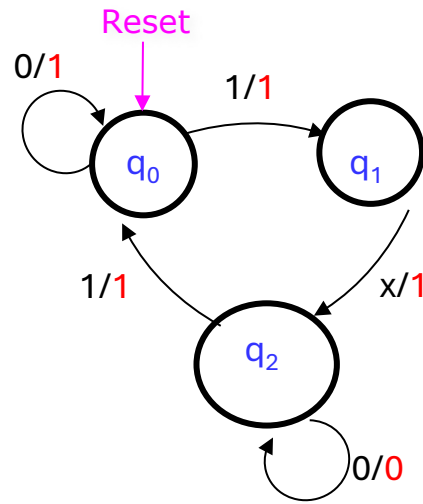
```
architecture behavioral of mealy is
type state_type is (q0,q1);
signal current_s,next_s: state_type;
begin

--combinational process to model next-state-logic
process (current_s,input_x)
begin
  case current_s is
    when q0 => --when current state is "q0"
      if(input_x = '0') then next_s <= q0;
      else next_s <= q1;
      end if;
    when q1 => --when current state is "q1"
      if(input_x = '0') then next_s <= q1;
      else next_s <= q0;
      end if;
  end case;
end process;
```

```
--sequential process to describe current-state-logic
process (clk,reset)
begin
  if (reset='1') then
    current_s <= q0; --asynchronous reset
  elsif (clk'event and clk='1') then
    current_s <= next_s; --state change.
  end if;
end process;
```

```
--combinational process to model output-logic
process (current_s,input_x)
begin
  case current_s is
    when q0 => --when current state is "q0"
      if(input_x = '0') then output_z <= '0';
      else output_z <= '1';
      end if;
    when q1 => --when current state is "q1"
      if(input_x = '0') then output_z <= '1';
      else output_z <= '0';
      end if;
  end case;
end process;
end behavioral;
```

Proposal: Coding in VHDL. Design the Following state machine



```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity state_machine is
    Port (
        clk      : in  std_logic;
        reset    : in  std_logic;
        inp      : in  std_logic;
        output   : out std_logic
    );
end state_machine;

```

```

architecture Behavioral of state_machine is

    type state_type is (q0, q1, q2);
    signal current_state, next_state:
        state_type;

begin

    -- State transition process
    process(clk, reset)
    begin
        if reset = '1' then
            current_state <= q0;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process;

```

```

-- Next state logic and output logic
process(current_state, inp)
begin
    case current_state is
        when q0 =>
            if inp = '0' then
                next_state <= q0;
                output <= '1';
            elsif inp = '1' then
                next_state <= q1;
                output <= '1';
            else
                next_state <= q0;
                output <= '1';
            end if;
        -- default case for q0
        when q1 =>
            next_state <= q2;
            output <= '1';
        when q2 =>
            if inp = '1' then
                next_state <= q0;
                output <= '1';
            elsif inp = '0' then
                next_state <= q2;
                output <= '0';
            else
                next_state <= q2;
                output <= '1';
            end if;
        -- default case for q2
        when others =>
            next_state <= q0;
            output <= '1';
        end case;
    end process;

end Behavioral;

```

Mealy Machine synthesis Block

Exercise: Explain The VHDL code shown below

```
ENTITY sumador_serie IS
PORT (a,b,clk, reset:
IN bit;
      suma: OUT bit);
END sumador_serie;
```

It represents a circuit that performs the addition of two numbers arriving serially on two different channels, from the bit with the smallest weight to the bit with the largest weight.

```
ARCHITECTURE para_sumador_serie OF
sumador_serie IS
  TYPE estados IS (q0,q1);
  SIGNAL estado, nxt_estado: estados;
  SIGNAL entradas: bit_vector(1 downto
0);
BEGIN
  entradas<=a&b;
sm: PROCESS(estado, entradas)
  BEGIN
    nxt_estado<=estado;
    CASE estado IS
      WHEN q0=>
        CASE entradas IS
          WHEN "11" =>
            nxt_estado<=q1;
            suma<='0';
          WHEN "00" =>
            nxt_estado<=q0;
            suma<='0';
          WHEN others =>
            nxt_estado<=q0;
            suma<='1';
          END CASE;
      WHEN q1=>
        CASE entradas IS
          WHEN "11" =>
            nxt_estado<=q1;
            suma<='1';
          WHEN "00" =>
            nxt_estado<=q0;
            suma<='1';
          WHEN others =>
            nxt_estado<=q1;
            suma<='0';
          END CASE;
        END CASE;
    END CASE;
  END PROCESS sm;
```

Mealy machine synthesis block

Exercise (CONT.)

1 What type of state machine does the VHDL code implement? Justify your answer.

Solution

1. Mealy machine since the output is a function of the state and the input. As you can see, the process sm that updates the outputs is triggered by the above signals (sm: PROCESS (state, inputs)).

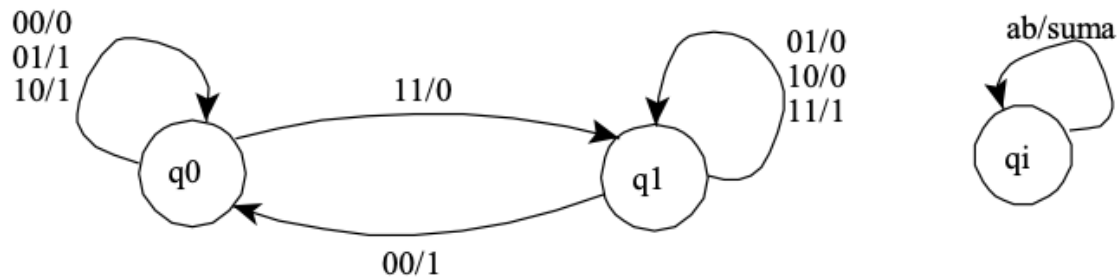
Mealy machine synthesis block

Exercise (CONT.)

2. Make the graph of the serial adder.

Solution

2.



3. Synthesise the circuit with basic logic gates and D-type bistables.

Solution:

Tabla de estados:

q(t)	q(t+1)/Z(t)			
	ab=00	ab=01	ab=10	ab=11
q0	q0/0	q0/1	q0/1	q1/0
q1	q0/1	q1/0	q1/0	q1/1

Con la siguiente tabla de asignación de estados se obtiene la tabla de transición:

Estado	y
q0	0
q1	1

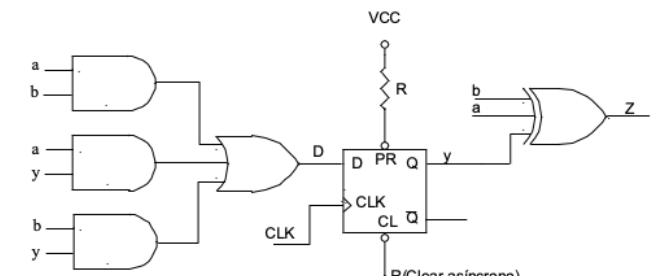
		q(t+1)				Z(t)			
q(t)	y	ab=00	ab=00	ab=00	ab=11	ab=00	ab=01	ab=10	ab=11
q0	0	0	0	1	1	0	1	1	0
q1	1	0	1	1	1	1	0	0	1
D									

Dado que la implementación se debe hacer con biestables tipo D las entradas de los mismos coinciden con las ecuaciones del próximo estado. Desarrollando los minterms de la tabla de transición y simplificando por Karnaugh queda:

$$D = \sum_3 (3,5,6,7) = ya + yb + ab$$

$$Z = \sum_3 (1,2,4,7) = y \oplus a \oplus b$$

Con lo que el circuito a diseñar será:



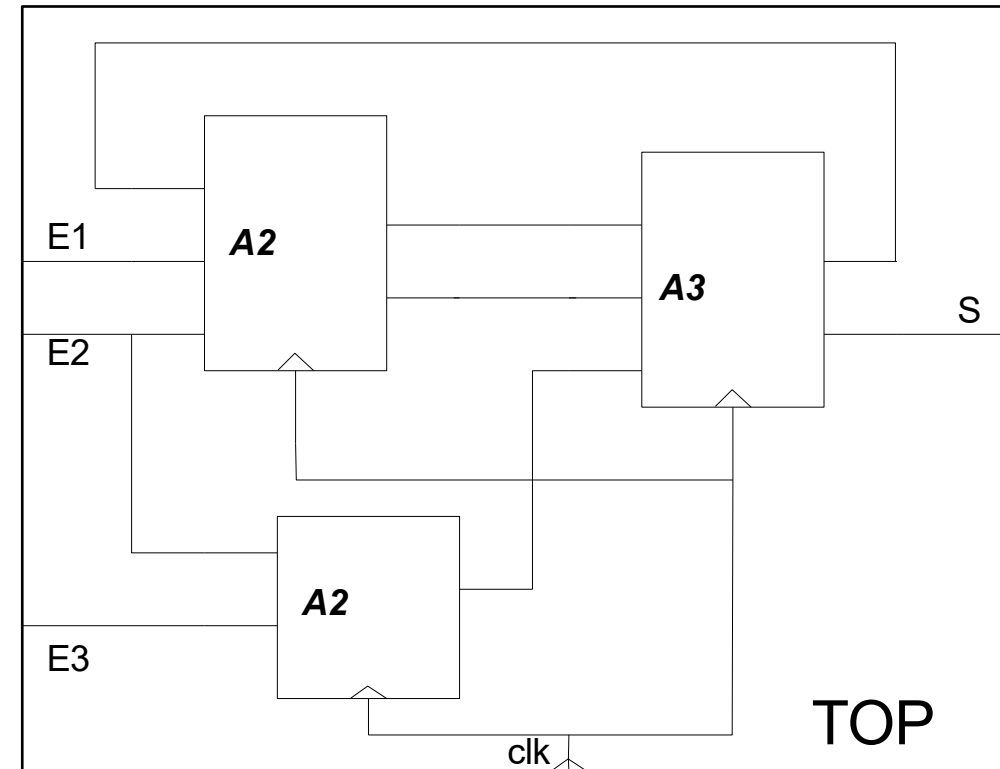
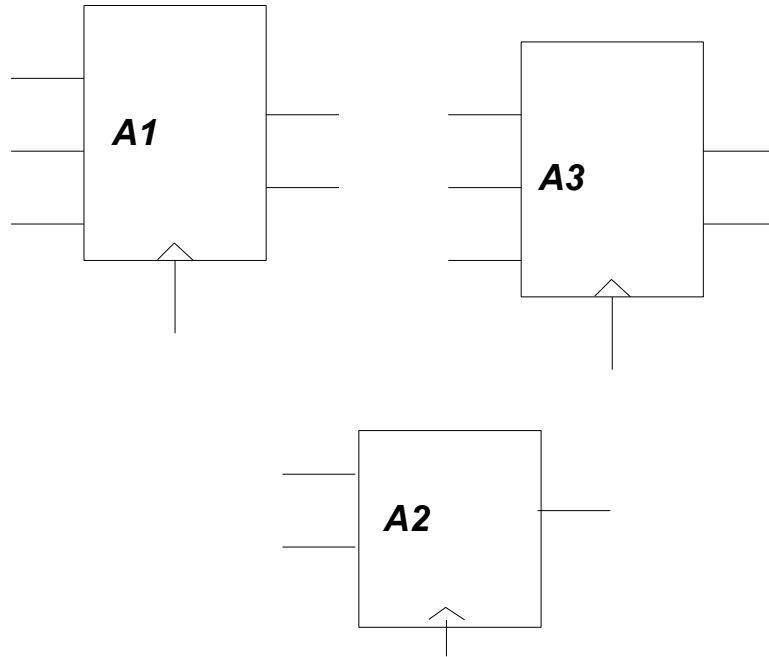
Simulation

To simulate a design it is necessary to generate stimuli for all inputs.

Some programs (older version of ISE Xilinx) allow you to generate the inputs interactively.

In most cases, it is necessary to write a description of the environment known as a test bench.

Example: Develop a Benchmark test design

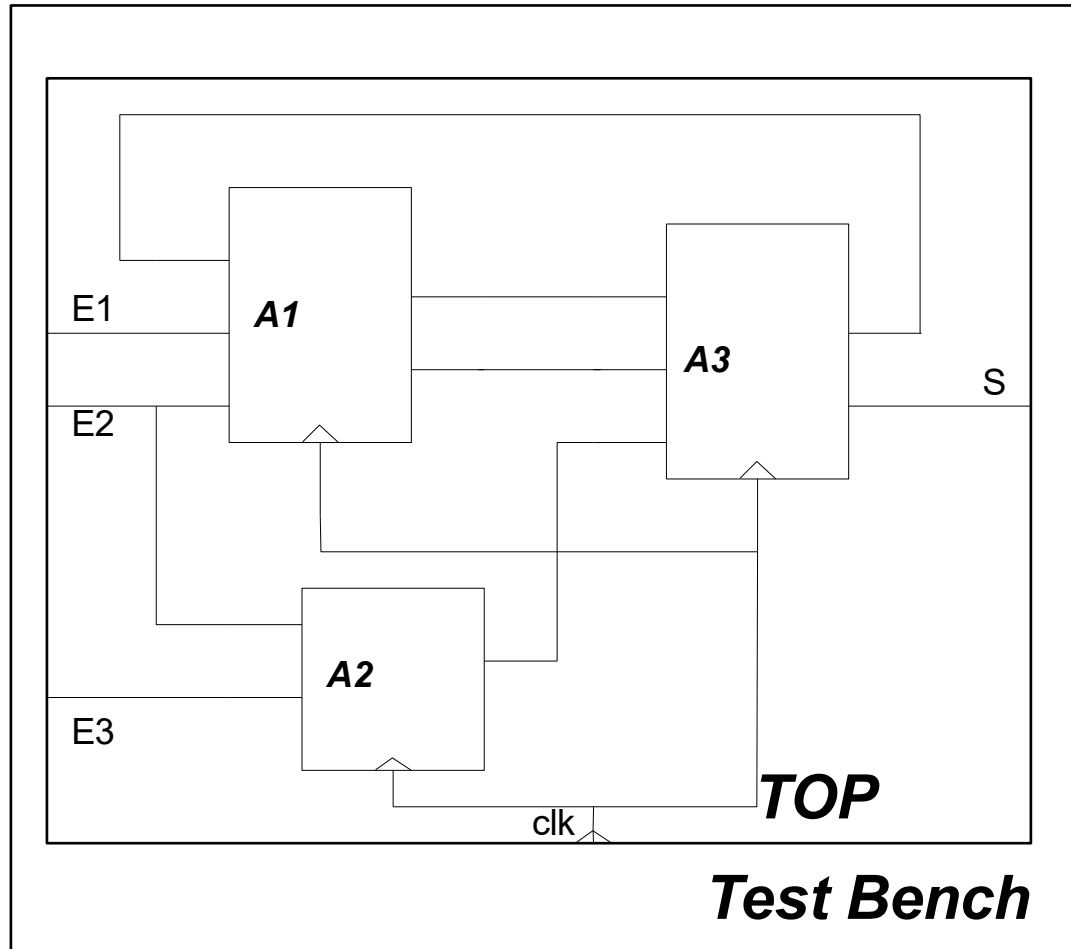


Blocks are connected to perform a structural block.

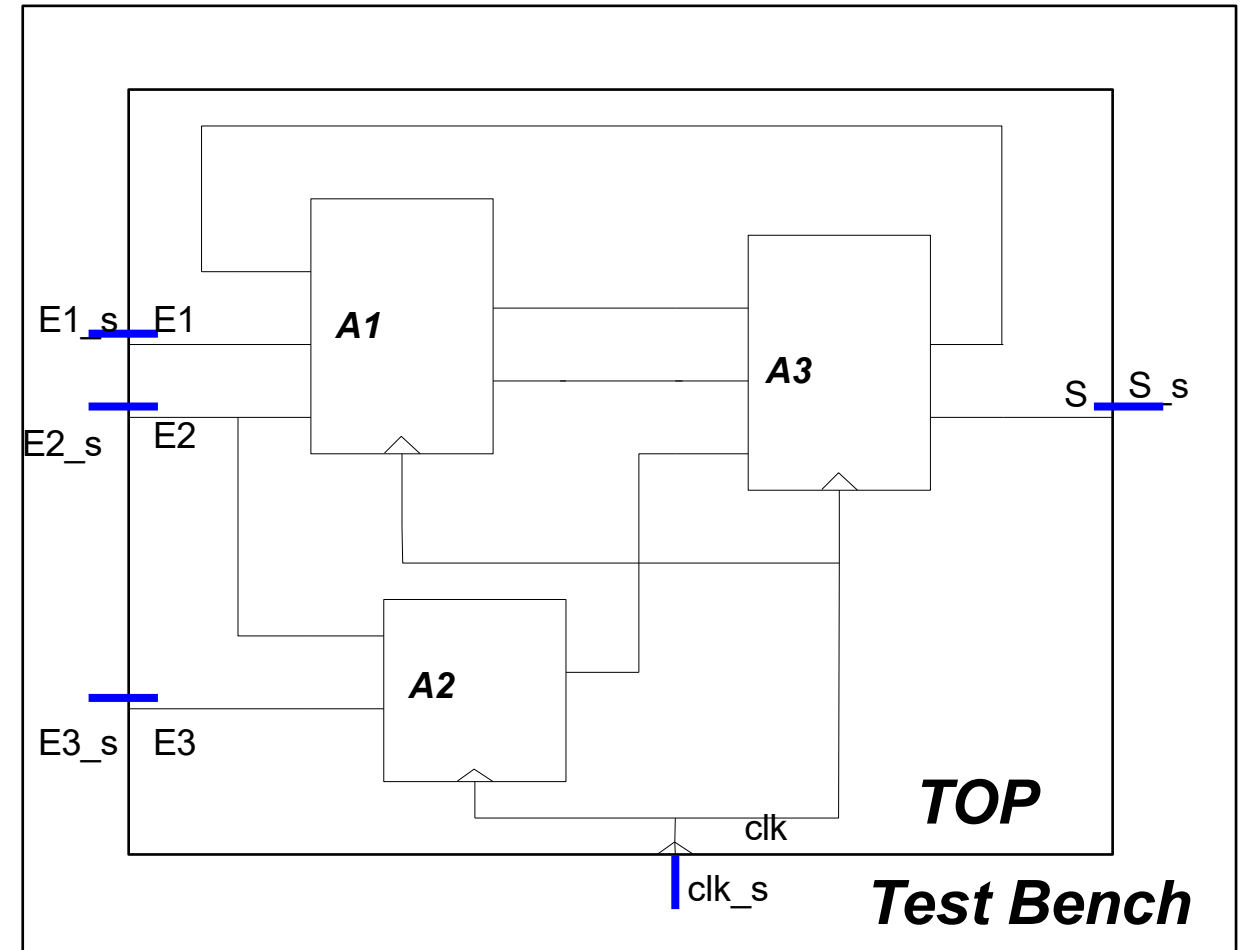
- Cables that do not go outside: signals
- Cables leading to the outside: ports

First, the basic building blocks are designed.

Example: Benchmark test design (CONT.)

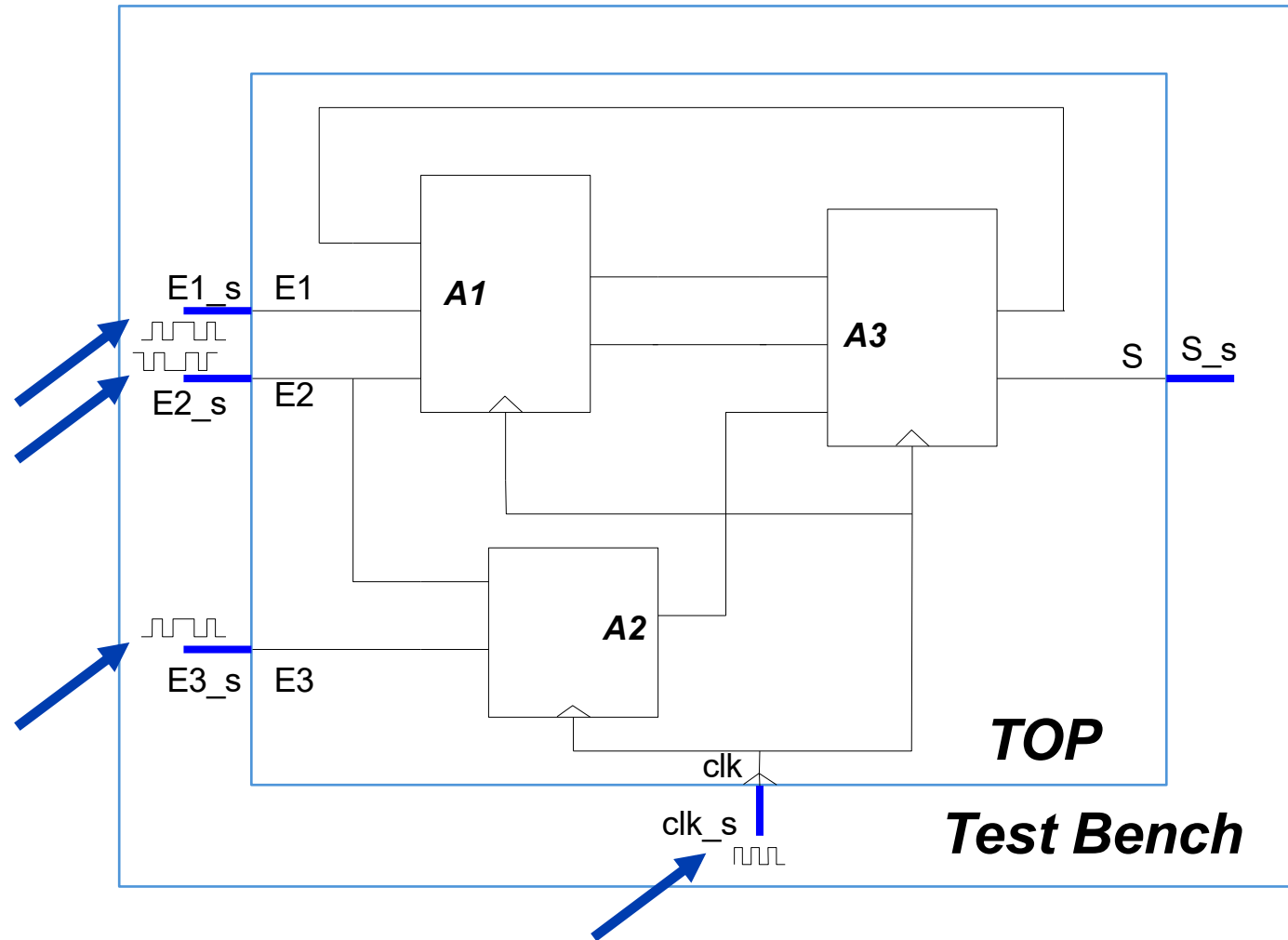


The design is included in an upper block without ports (the TestBench).



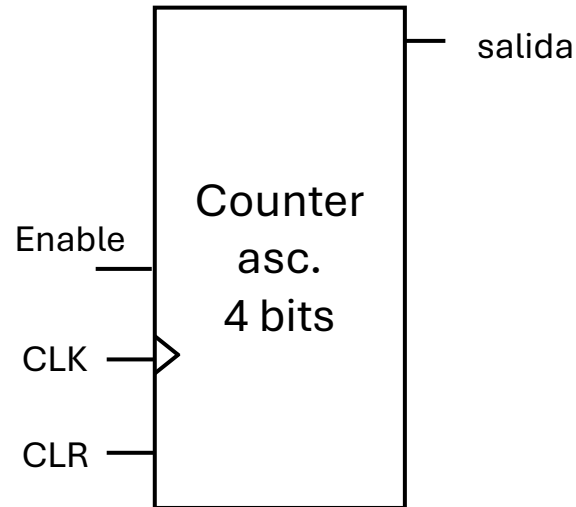
One signal is declared for each of the ports.

Example: Benchmark test design (CONT.)



In the architecture of the testbench, stimuli are assigned to the inputs.

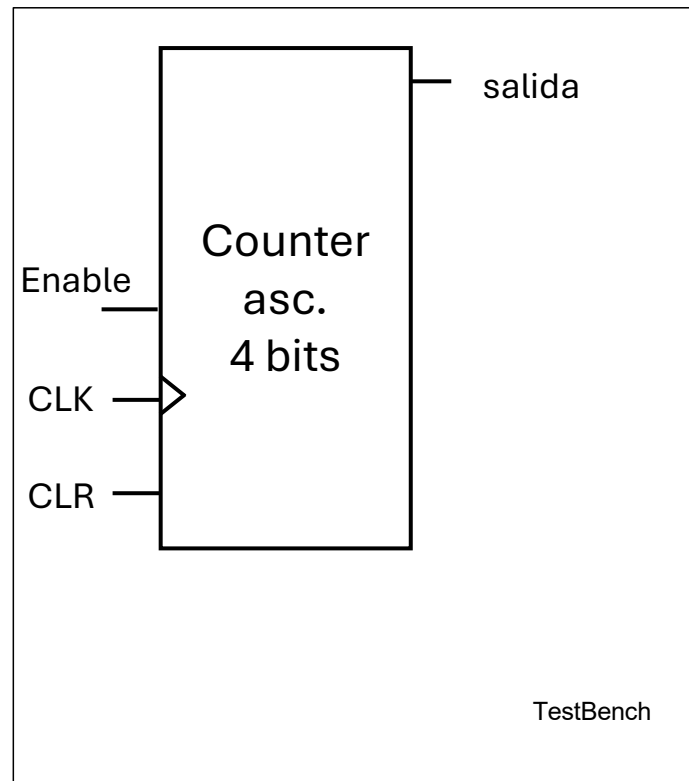
Example: Counter testbench



```
ENTITY contador
IS PORT (
  salida: out std_logic_vector(7 DOWNTO 0);
  clk: in std_logic;
  reset: in
    std_logic;
  enable: in
    std_logic
);
END contador;
```

```
Architecture contador_arch of contador is
  signal cuenta, p_cuenta: std_logic_vector(7 downto 0);
begin
  salida<=cuenta;
  comb:process(cuenta, enable)
  Begin
    if enable='1' then
      p_cuenta<=cuenta+1;
    Else
      p_cuenta<=cuenta;
    end if;
  end process;
  sinc:process(clk, reset)
  begin
    if reset='1' then
      cuenta <= (others => '0');
    elsif(clk='1' and clk'event) then
      cuenta <= p_cuenta;
    end if;
  end process;
End contador_arch;
```

Example: Counter testbench (CONT.)



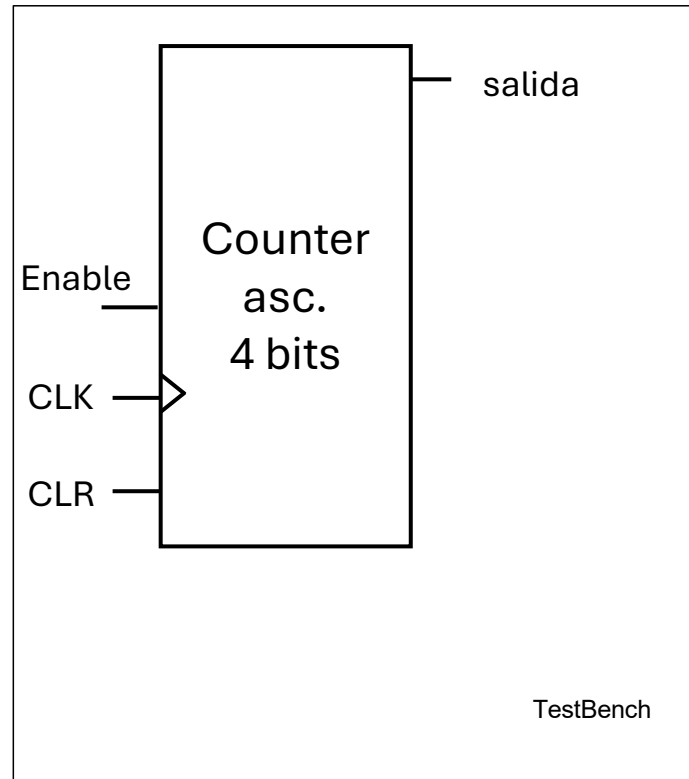
```
ENTITY    TestBench    IS  
END    TestBench;
```

STEP 1: Define TestBench entity without adding any ports

```
ARCHITECTURE    TestBench_arch OF TestBench IS  
COMPONENT contador  
PORT (  
    salida: out std_logic_vector(7 DOWNTO 0);  
    clk: in std_logic;  
    reset: in  
        std_logic; enable:  
        in std_logic  
);  
END COMPONENT;
```

STEP 2: Declare the circuit to be simulated as a component

Example: Counter testbench (CONT.)



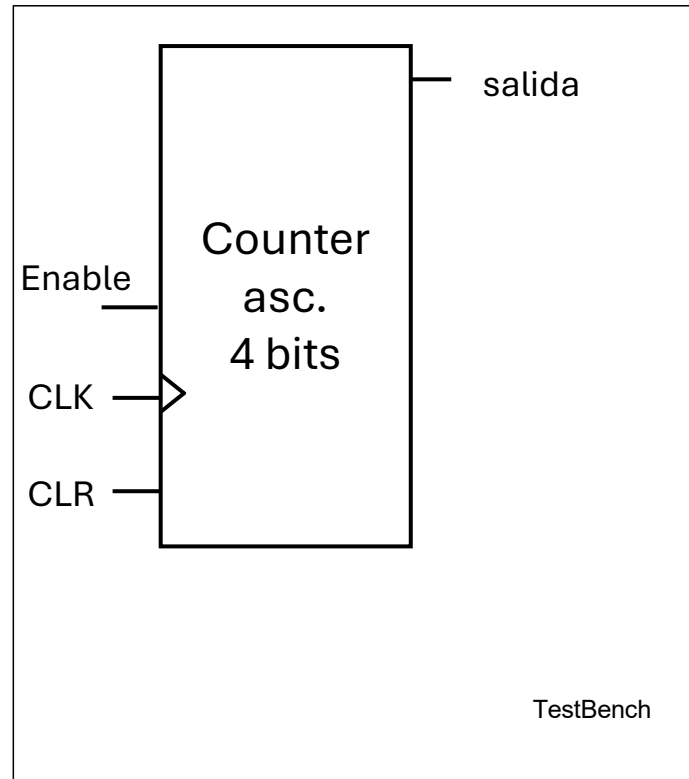
Default value (time 0)

```
SIGNAL salida_s : std_logic_vector (7 downto 0);  
SIGNAL clk_s : std_logic := '0'; SIGNAL  
enable_s : std_logic; SIGNAL reset_s :  
std_logic;
```

```
BEGIN - - content of the architecture
```

STEP 3: Declare one signal per component port.

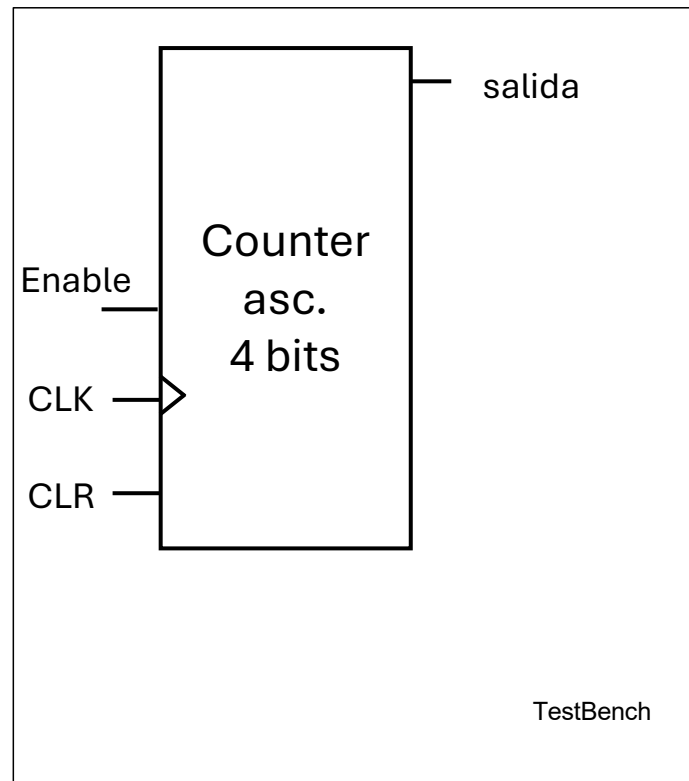
Example: Counter testbench (CONT.)



```
DUT: contador
PORT MAP (
    salida => salida_s,
    clk => clk_s,
    enable => enable_s,
    reset => reset_s);
```

STEP 4: Make an instance of the component, connecting its ports to the corresponding signals.

Example: Counter testbench



```
clk_s <= NOT clk_s AFTER 50 ns;
```

```
estimulos: PROCESS
```

```
BEGIN
```

```
    reset_s<='1';
```

```
    enable_s<='1';
```

```
    WAIT FOR 100 ns;
```

```
    reset_s<='0';
```

```
    WAIT FOR 1000 ns;
```

```
    enable_s<='0';
```

```
    WAIT;
```

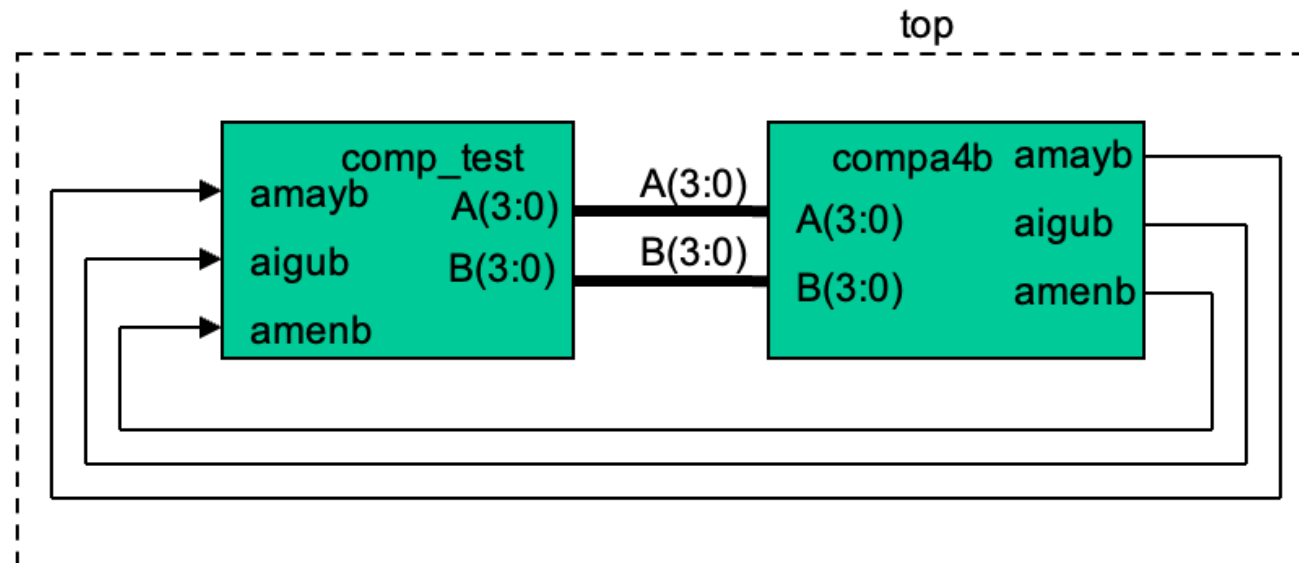
```
END PROCESS;
```

```
END TestBench_arch;
```

STEP 5: Generate the stimuli

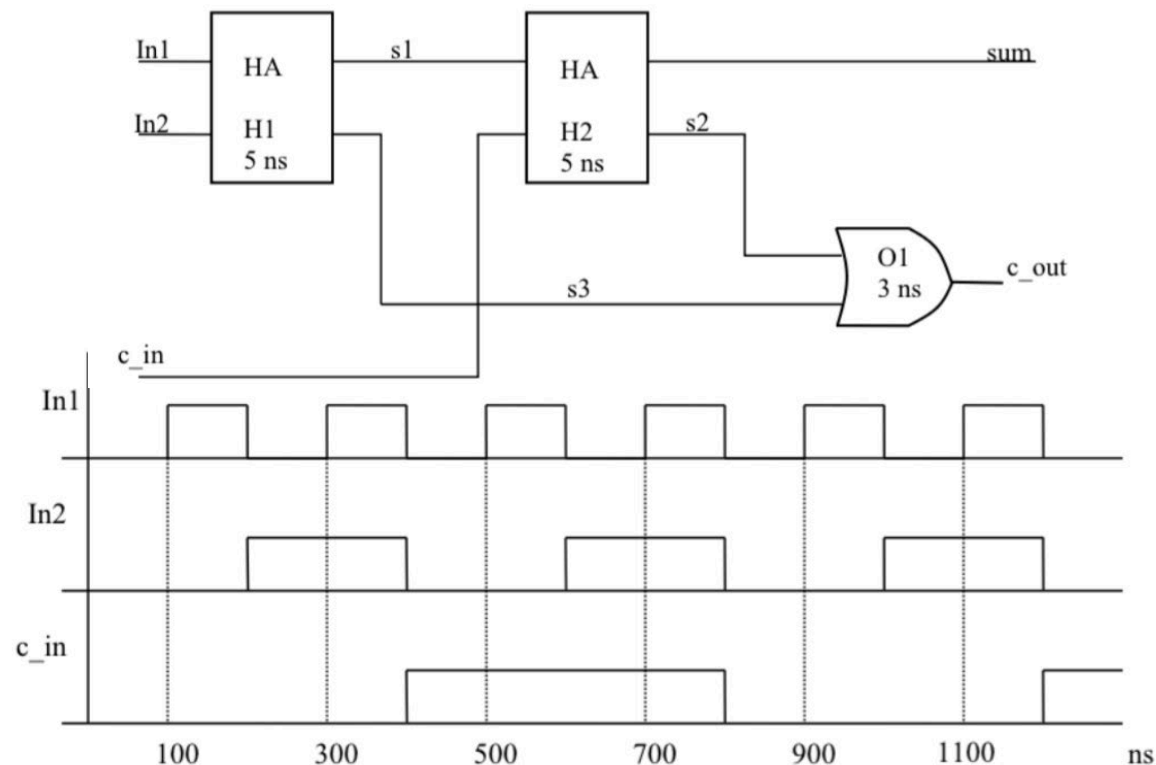
TESTBENCH

Exercise: A test bench for a 4-bit comparator has been implemented using test vectors stored in a table, as shown in the following VHDL code. The hierarchical level of the code is shown in figure. The `to_string` function defined in the `std_iopak` package of the `std_developerskit` library has been used. In this library there are different functions defined for debugging. Specifically, the function used transforms the value passed as an index into a string that is displayed on the screen. Indicate the messages displayed on the screen when the given code is simulated.

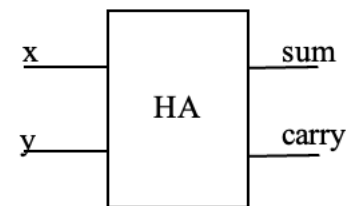


TESTBENCH

Exercise: Design a half-adder (HA) based full adder circuit as shown in Figure. Taking into account that the input-output data are of bit type, implement an entity and a structural level architecture for it. Also, define a test bench for the designed model that generates stimuli as shown in Figure and obtain the circuit outputs for these stimuli.



The first thing to design is the components of the total adder (HA, O1) using an RTL model. In the case of the half adder, the following equations are obtained from its truth table and by developing the Minterms:



$$\begin{aligned} \text{sum} &= x \oplus y \\ \text{carry} &= x \cdot y \end{aligned}$$

Figure

Accessing files

Built-in text file I/O capabilities provided by the IEEE standard libraries.

The basic libraries contain simple functions for file access.

It is possible to read stimuli from files

It is possible to save results to files

```
LIBRARY STD;  
USE STD.TEXTIO.ALL LIBRARY IEEE;  
USE IEEE.STD_LOGIC_TEXTIO.ALL
```

Syntax change in 1993!

Original Syntax

```
FILE fin : TEXT IS IN "i.txt";  
FILE fout: TEXT IS OUT "o.txt";
```

VHDL'93 Syntax

```
FILE fin : TEXT open READ_MODE is "i.txt";  
FILE fout : TEXT open WRITE_MODE is "o.txt";
```

Accessing files

Example: Reading from and Writing to a File in VHDL

Libraries

First, ensure that you include the necessary libraries:

```
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
use IEEE.STD_LOGIC_TEXTIO.ALL;  
use std.textio.all;
```

File Declaration

You declare a file and specify whether it's for reading or writing:

```
file input_file  : text open read_mode is "input.txt";  
file output_file : text open write_mode is  
"output.txt";
```

Reading from a File

You can read data from a file using the readline and read procedures:

```
variable line_in : line;  
variable data_in : integer;  
begin  
    if not endfile(input_file) then  
        readline(input_file, line_in);  
        read(line_in, data_in);  
    end if;
```

Accessing files

Example: Reading from and Writing to a File in VHDL Libraries

Writing to a File

Similarly, you can write data to a file using the writeline and write procedures:

```
variable line_out : line;
variable data_out : integer :=
42;
begin
    write(line_out, data_out);
    writeline(output_file,
line_out);
```

```
architecture Behavioral of FileExample is
    file input_file : text open read_mode is
"input.txt";
    file output_file : text open write_mode is
"output.txt";
    signal done : boolean := false;
begin
    process
        variable line_in : line;
        variable line_out : line;
        variable data_in : integer;
        variable data_out : integer;
    begin
        while not endfile(input_file) loop
            readline(input_file, line_in);
            read(line_in, data_in);
            data_out := data_in + 1;
            write(line_out, data_out);
            writeline(output_file, line_out);
        end loop;
        done <= true;
        wait;
    end process;
end Behavioral;
```

Accessing files

You can only read entire lines

Line variable: LINE; Readline (end, line);

You can pull the fields out of the line one by one:

Read(line, primer_campo); Read(line, segundo_campo);

Fields written in decimal or format

1 2

3 4

16#FF# 1

File writing

You can only write whole lines

Line variable: LINE; writeline (fout, linea);

The line is built field by field

```
write(line, primer_campo); write(line, " ");
```

```
write(line, segundo_campo);
```

Fields are written in their natural format

There are usually HWRITE functions for writing to HEX and BWRITE for writing to binary.

Accessing files

Example

```
PROCESS (load)
VARIABLE lineain, lineout: LINE;
VARIABLE campol, campo2: integer;
BEGIN
    IF (load = '1') THEN
        READLINE(fin, lineain);
        READ(lineain, campol);
        READ(lineain, campo2);
        entrada1 <= campol;
        entrada2 <= campo2;
    END IF;
    campol := conv_integer(senal);
    WRITE(lineout, campol);
    WRITELINE(fout, lineout);
END PROCESS;
```

Accessing files

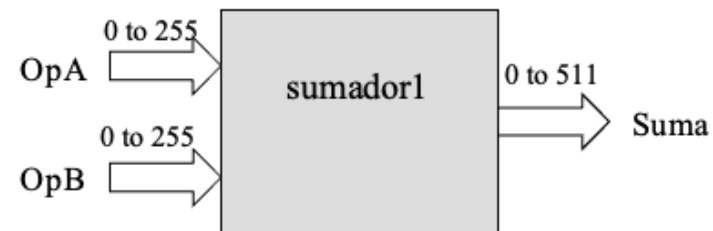
Exercise: It is desired to design an adder for integers between 0 and 255 with a data flow description level according to the scheme in Figure. To check its operation, a test bench shall be used by reading-writing external files. Test vectors shall be introduced through a file named Data1.txt and the results shall be obtained in another file named Results1.txt according to the format indicated:

Datos1.txt

53	52
-32	345
34	-32
20	20
0	1

Resultados1.txt

53+52=105
-32+345=313
34-32=2
20+20=40
0+1=1

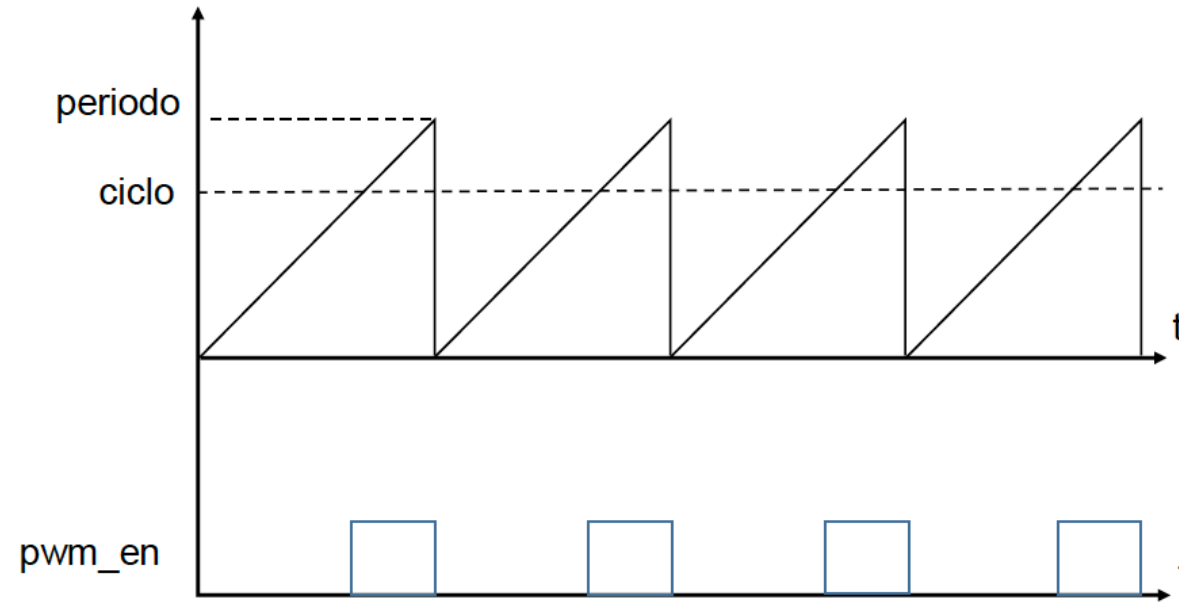


Figure

Note that the "+/-" signs must be added depending on whether the second operand is positive or negative and the "=" sign before writing the result of the addition. The coding shall be done using VHDL-93.

PEI 19-20

Se desea llevar a cabo la implementación de un módulo capaz de generar una señal PWM. Esta señal, muy común en las aplicaciones de potencia, consiste en la comparación de un valor ciclo con una señal en diente de sierra, de forma que cuando la señal se encuentre por encima del valor ciclo, se active la señal pwm_en. La figura muestra gráficamente y de forma esquemática la generación de dicha señal.



De forma general, el diseño consta de una señal de reloj `clk`, activa en flanco de subida, y una señal de reset `rst`, asíncrona y activa a nivel alto. Para ello, el proceso de diseño se divide en los siguientes apartados:

a) En primer lugar, se debe especificar en VHDL sintetizable la arquitectura para el módulo `rampa_gen`, para el cual se proporciona la entidad siguiente. En ella, se debe generar la señal en diente de sierra en el puerto `rampa`, a partir de su valor máximo definido por la entrada `periodo`. Recuerde incluir las librerías que considere necesarias.

PEI 19-20 (CONT.)

a) En primer lugar, se debe especificar en VHDL sintetizable la arquitectura para el módulo rampa_gen, para el cual se proporciona la entidad siguiente. En ella, se debe generar la señal en diente de sierra en el puerto rampa, a partir de su valor máximo definido por la entrada periodo. Recuerde incluir las librerías que considere necesarias.

```
entity rampa_gen is
  port (
    rst    : in std_logic;
    clk    : in std_logic;
    periodo : in std_logic_vector(15 downto 0);
    rampa   : out std_logic_vector(15 downto 0));
end rampa_gen;
```

PEI 19-20 (CONT.)

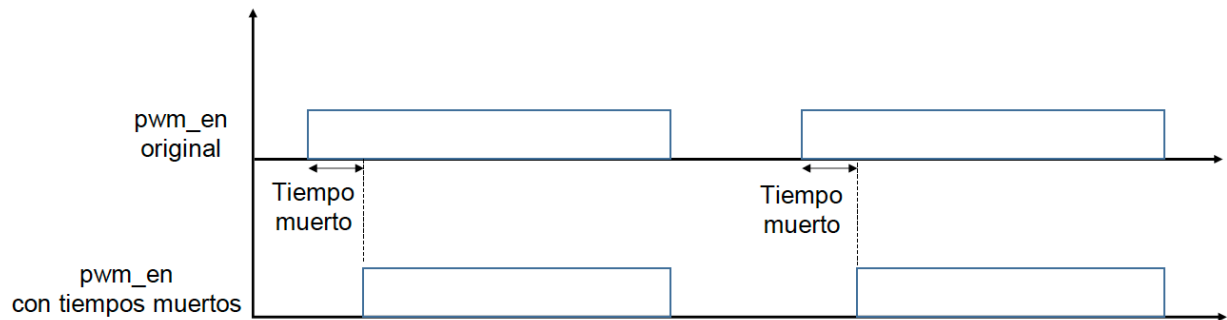
b) A partir del módulo `rampa_gen` (incluido como un componente), especifique en VHDL sintetizable la arquitectura de la entidad `pwm_gen`, proporcionada a continuación. En ella, el puerto `ciclo` establece el valor de comparación para generar la salida `pwm_en`, según la figura anterior. Por otro lado, la salida `pwm_en_neg` es una copia negada de la salida `pwm_en`. Recuerde incluir las librerías que considere necesarias. **(20 puntos)**

```
entity pwm_gen is
  port (
    rst      : in std_logic;
    clk      : in std_logic;
    periodo  : in std_logic_vector(15 downto 0);
    ciclo    : in std_logic_vector(15 downto 0);
    pwm_en   : out std_logic;
    pwm_en_neg : out std_logic);
end pwm_gen;
```

```
component rampa_gen
  port (
    rst : in std_logic;
    clk : in std_logic;
    periodo : in std_logic_vector(15 downto 0);
    rampa : out std_logic_vector(15 downto 0));
end component;
```

PEI 19-20 (CONT.)

c) Como elemento de protección, la mayoría de los generadores de PWM llevan un módulo de generación de tiempos muertos que impiden que las salidas `pwm_en` y `pwm_en_neg` puedan activarse a nivel alto simultáneamente. Para ello, cuando una de estas salidas debe pasar a nivel alto, se introduce un número de ciclos de reloj de espera (en el diseño especificados por el genérico `TiempoMuerto`). A partir de la arquitectura obtenida en el apartado anterior, y modificando la entidad para incluir el nuevo genérico, especifique la nueva arquitectura capaz de generar los correspondientes tiempos muertos, según se muestra en la figura adjunta. Nótese que los ciclos de espera en la activación de las señales de PWM sólo afectan en su flanco de subida. **(40 puntos)**



```
entity pwm_gen is
  generic (
    TiempoMuerto : integer);
  port (
    rst          : in std_logic;
    clk          : in std_logic;
    periodo      : in std_logic_vector(15 downto 0);
    ciclo        : in std_logic_vector(15 downto 0);
    pwm_en       : out std_logic;
    pwm_en_neg   : out std_logic);
end pwm_gen;
```

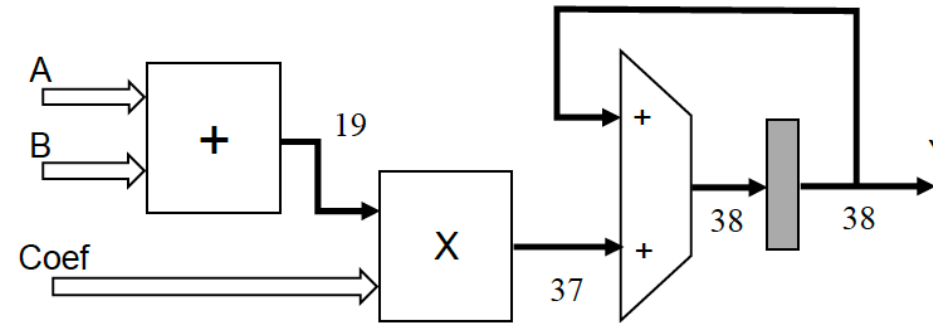
PEI 19-20 (CONT.)

d) Realice un testbench (banco de pruebas) especificado en VHDL para poder verificar el funcionamiento de la entidad `pwm_gen`, si se desea configurar un periodo de 86 ciclos de la señal de reloj, un ciclo de 50 ciclos de reloj, y un tiempo muerto (`TiempoMuerto`) de 5 ciclos de la señal reloj. La señal de reloj tendrá una frecuencia de 100 MHz. Defina las librerías empleadas, entidad y arquitectura del testbench. **(20 puntos)**

PEI 17-18

Exercise 1

Se desea llevar a cabo la implementación de un filtro FIR con simetría de coeficientes, para lo cual se empieza por la especificación del módulo *BOP*, básico para el cálculo del mismo. Dicho módulo tiene un esquema como el mostrado en la figura, con una señal de reset asíncrono *mrst* activa a nivel alto, y un reloj *mclk* activo en flanco ascendente. Tanto los datos de entrada (*A*, *B* y *Coef*) como los de salida (*Y*) tienen sus correspondientes señales de validación, *din_vld* y *dout_vld* respectivamente, activas a nivel alto durante un periodo de la señal de reloj *mclk*.



- a) Determine sobre la figura el ancho de palabra que será necesario en cada bus del esquema para evitar el desbordamiento y/o truncamiento, en el caso particular en que los datos de entrada *A* y *B* tienen *nBitsData*=18 bits, mientras que los coeficientes *Coef* también tienen *nBitsCoef*=18 bits.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_signed.all;
use ieee.std_logic_arith.all;

entity BOP is

    generic (
        nBitsData : integer := 18;
        nBitsCoef : integer := 18);

    port (
        mclk   : in std_logic;
        mrst   : in std_logic;
        A      : in std_logic_vector(nBitsData-1 downto 0);
        B      : in std_logic_vector(nBitsData-1 downto 0);
        din_vld : in std_logic;
        Coef   : in std_logic_vector(nBitsCoef-1 downto 0);
        Y      : out std_logic_vector(nBitsData+nBitsCoef+1 downto 0);
        dout_vld : out std_logic);

end BOP;
```

PEI 17-18 Exercise 1 (CONT.)

- b) Realice la especificación de la arquitectura correspondiente a la entidad *BOP*, empleando VHDL sintetizable. No olvide reflejar qué librerías y paquetes usará para la definición de la arquitectura. Para el diseño del multiplicador, deberá emplear el componente externo *MULT25x18*, descrito a continuación. Recuerde manejar el ancho de bus de las distintas señales internas a partir de los genéricos *nBitsData* y *nBitsCoef*, definidos en la entidad para hacer la misma escalable.

